In [1]:
```
import pandas as pd
```

https://www.kaggle.com/datasets/noir1112/e-commerce-customer-engagement

In [2]:
```
df =  pd.read_csv(r"C:\Users\mundr\Downloads\archive\ecommerce_customer_d
df
```

Out[2]:

	CustomerID	RegistrationDate	Age	Gender	IncomeLevel	Country	City
0	NaN	2020-05-10	25.0	NaN	NaN	Other	Tokyo
1	CUST00002	2021-07-18	NaN	Prefer not to say	High	Germany	London
2	CUST00003	2021-02-04	43.0	NaN	High	France	NaN
3	CUST00004	2020-12-31	49.0	Female	High	Australia	Los Angeles
4	CUST00005	2022-06-27	29.0	Female	Very High	Australia	Tokyo
...	...	...	...	...	...	...	...
9995	CUST09996	2018-09-29	39.0	Female	High	Japan	New York
9996	CUST09997	2020-10-25	58.0	Prefer not to say	Low	France	Sydney
9997	CUST09998	2023-04-07	14.0	Other	Very High	USA	New York
9998	CUST09999	2021-05-27	29.0	Other	High	Canada	Berlin
9999	CUST10000	2021-02-15	32.0	NaN	Low	France	Paris

10000 rows × 23 columns

1.Business Problem

- To analyze customer data and understand their buying behavior, spending, satisfaction, and returns so that the business can improve sales and customer satisfaction.

2. Data Understanding

In [3]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 23 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   CustomerID                   9508 non-null   str
 1   RegistrationDate             9504 non-null   str
 2   Age                          9485 non-null   float64
 3   Gender                       7388 non-null   str
 4   IncomeLevel                  7497 non-null   str
 5   Country                      9507 non-null   str
 6   City                         9517 non-null   str
 7   TotalPurchases               9470 non-null   float64
 8   AverageOrderValue            9481 non-null   float64
 9   CustomerLifetimeValue        9507 non-null   float64
 10  FavoriteCategory             8411 non-null   str
 11  SecondFavoriteCategory       8450 non-null   str
 12  EmailEngagementRate          9524 non-null   float64
 13  SocialMediaEngagementRate    9472 non-null   float64
 14  MobileAppUsage               7543 non-null   str
 15  CustomerServiceInteractions  9482 non-null   float64
 16  AverageSatisfactionScore     9504 non-null   float64
 17  EmailConversionRate          9477 non-null   float64
 18  SocialMediaConversionRate    9506 non-null   float64
 19  SearchEngineConversionRate   9495 non-null   float64
 20  RepeatCustomer               9525 non-null   str
 21  PremiumMember                9506 non-null   str
 22  HasReturnedItems             9471 non-null   str
dtypes: float64(11), str(12)
memory usage: 1.8 MB
```

In [4]:
```
df.describe()
```

Out[4]:

	Age	TotalPurchases	AverageOrderValue	CustomerLifetimeValue	Er
count	9485.000000	9470.000000	9481.000000	9507.000000	
mean	34.536742	5.044773	189.224562	672.919948	
std	15.117146	3.205844	1472.277418	5729.439186	
min	-25.000000	0.000000	1.235282	-9331.077011	
25%	24.000000	3.000000	27.441273	95.662915	
50%	35.000000	5.000000	54.532801	248.018238	
75%	45.000000	7.000000	109.260008	545.787253	
max	91.000000	27.000000	51810.123750	420810.815641	

In [5]:
```
df.columns
```

Out[5]:
```
Index(['CustomerID', 'RegistrationDate', 'Age', 'Gender', 'IncomeLevel',
       'Country', 'City', 'TotalPurchases', 'AverageOrderValue',
       'CustomerLifetimeValue', 'FavoriteCategory', 'SecondFavoriteCateg
ory',
       'EmailEngagementRate', 'SocialMediaEngagementRate', 'MobileAppUsa
ge',
       'CustomerServiceInteractions', 'AverageSatisfactionScore',
       'EmailConversionRate', 'SocialMediaConversionRate',
       'SearchEngineConversionRate', 'RepeatCustomer', 'PremiumMember',
       'HasReturnedItems'],
      dtype='str')
```

3. Data Cleaning

In [6]:
```
df.isnull().mean() * 100
```

Out[6]:
```
CustomerID                   4.92
RegistrationDate             4.96
Age                          5.15
Gender                      26.12
IncomeLevel                 25.03
Country                      4.93
City                         4.83
TotalPurchases               5.30
AverageOrderValue            5.19
CustomerLifetimeValue        4.93
FavoriteCategory            15.89
SecondFavoriteCategory      15.50
EmailEngagementRate          4.76
SocialMediaEngagementRate    5.28
MobileAppUsage              24.57
CustomerServiceInteractions  5.18
AverageSatisfactionScore     4.96
EmailConversionRate          5.23
SocialMediaConversionRate    4.94
SearchEngineConversionRate   5.05
RepeatCustomer               4.75
PremiumMember                4.94
HasReturnedItems             5.29
dtype: float64
```

In [7]:
```
num = df.select_dtypes(include =  "number").columns
num
```

Out[7]:
```
Index(['Age', 'TotalPurchases', 'AverageOrderValue', 'CustomerLifetimeVa
lue',
       'EmailEngagementRate', 'SocialMediaEngagementRate',
       'CustomerServiceInteractions', 'AverageSatisfactionScore',
       'EmailConversionRate', 'SocialMediaConversionRate',
       'SearchEngineConversionRate'],
      dtype='str')
```

In [8]:
```
cat = df.select_dtypes(include = "object").columns
cat
```

```
C:\Users\mundr\AppData\Local\Temp\ipykernel_13220\4209226523.py:1: Pandas4
Warning: For backward compatibility, 'str' dtypes are included by select_d
types when 'object' dtype is specified. This behavior is deprecated and wi
ll be removed in a future version. Explicitly pass 'str' to `include` to s
elect them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#str
ing-migration-select-dtypes for details on how to write code that works wi
th pandas 2 and 3.
  cat = df.select_dtypes(include = "object").columns
```

Out[8]:
```
Index(['CustomerID', 'RegistrationDate', 'Gender', 'IncomeLevel', 'Count
ry',
       'City', 'FavoriteCategory', 'SecondFavoriteCategory', 'MobileAppU
sage',
       'RepeatCustomer', 'PremiumMember', 'HasReturnedItems'],
      dtype='str')
```

In [9]:
```
for i in num:
    df[i] = df[i].fillna(df[i].median())
```

In [10]:
```
for j in cat:
        df[j] = df[j].fillna(df[j].mode()[0])
```

In [11]:
```
df.isnull().mean() * 100
```

Out[11]:
```
CustomerID                    0.0
RegistrationDate              0.0
Age                           0.0
Gender                        0.0
IncomeLevel                   0.0
Country                       0.0
City                          0.0
TotalPurchases                0.0
AverageOrderValue             0.0
CustomerLifetimeValue         0.0
FavoriteCategory              0.0
SecondFavoriteCategory        0.0
EmailEngagementRate           0.0
SocialMediaEngagementRate     0.0
MobileAppUsage                0.0
CustomerServiceInteractions   0.0
AverageSatisfactionScore      0.0
EmailConversionRate           0.0
SocialMediaConversionRate     0.0
SearchEngineConversionRate    0.0
RepeatCustomer                0.0
PremiumMember                 0.0
HasReturnedItems              0.0
dtype: float64
```

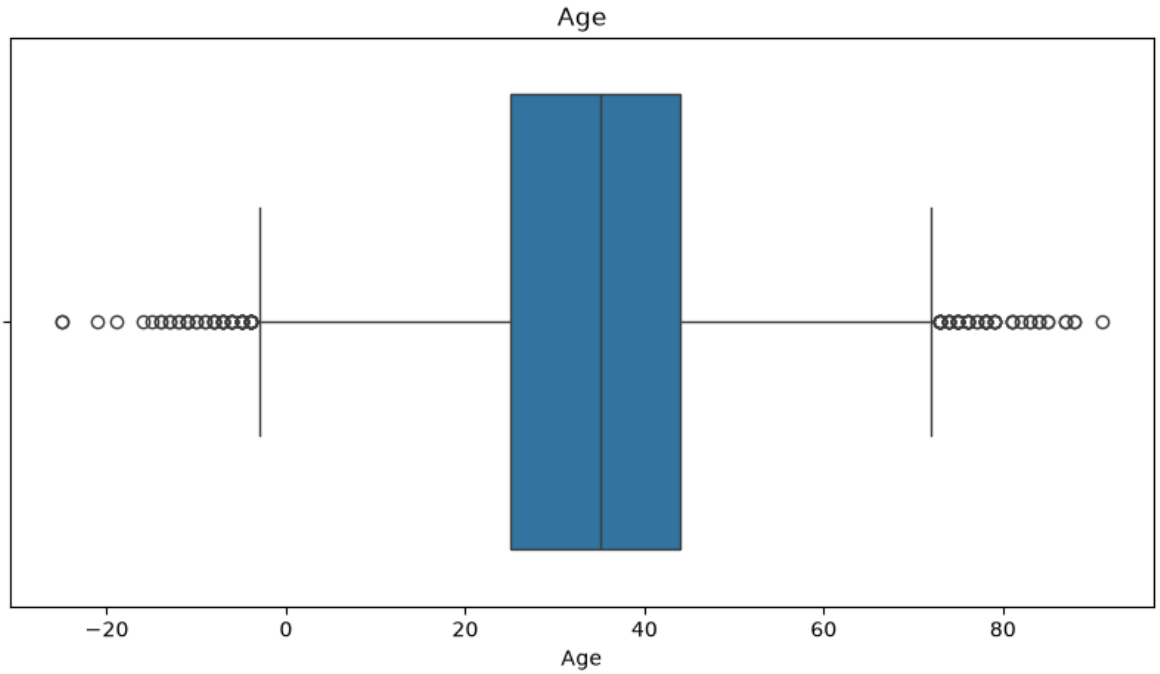
In [12]:
```
import matplotlib.pyplot as plt
```

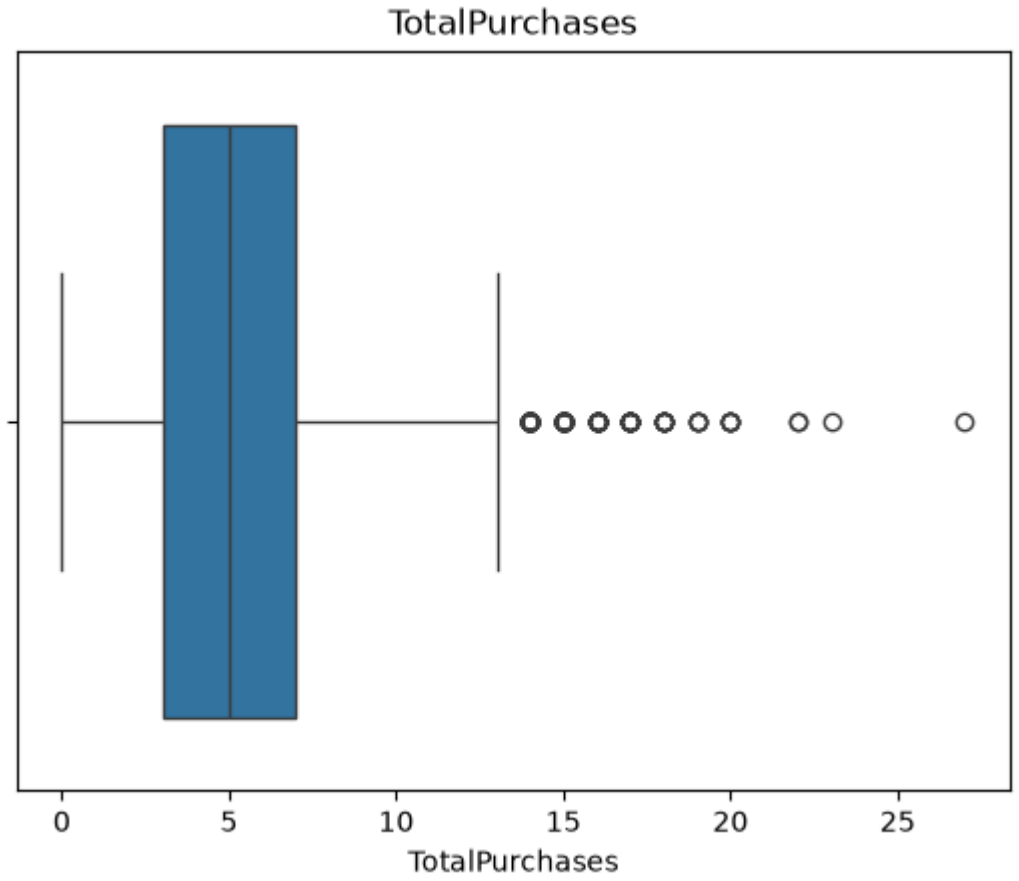
In [13]:
```
import seaborn as sns
```

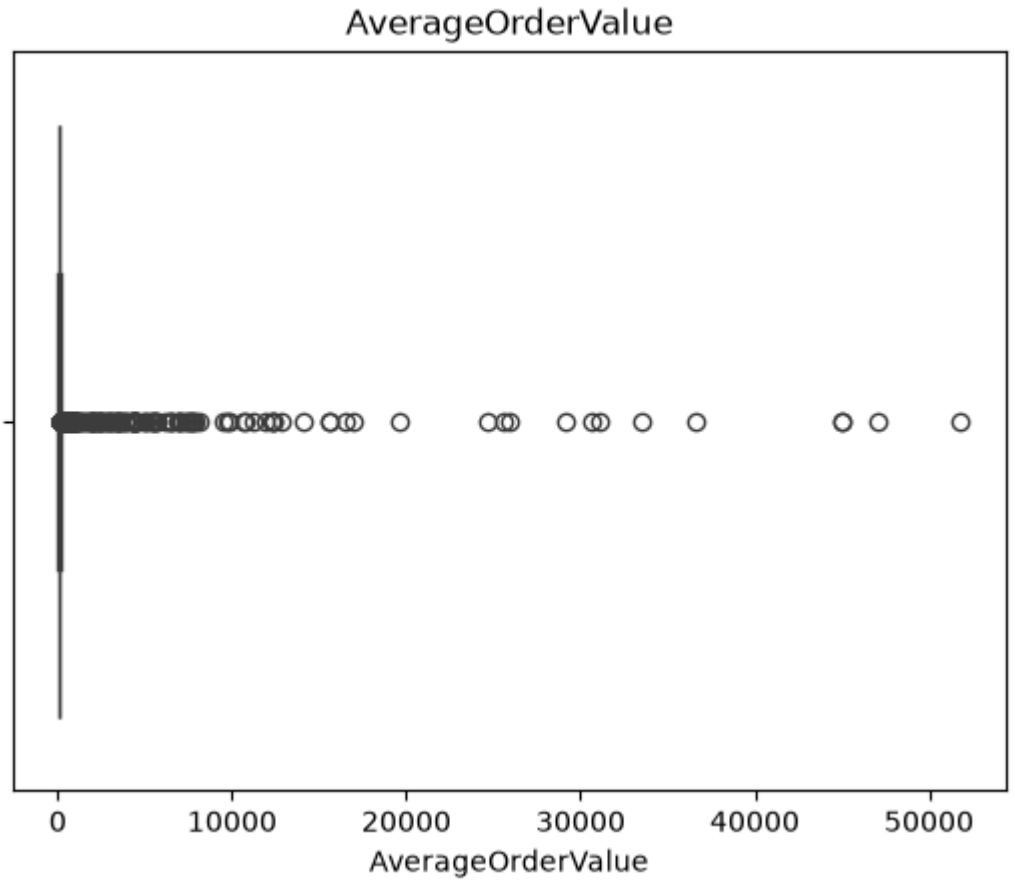
4.Checking Outliers for numerical columns

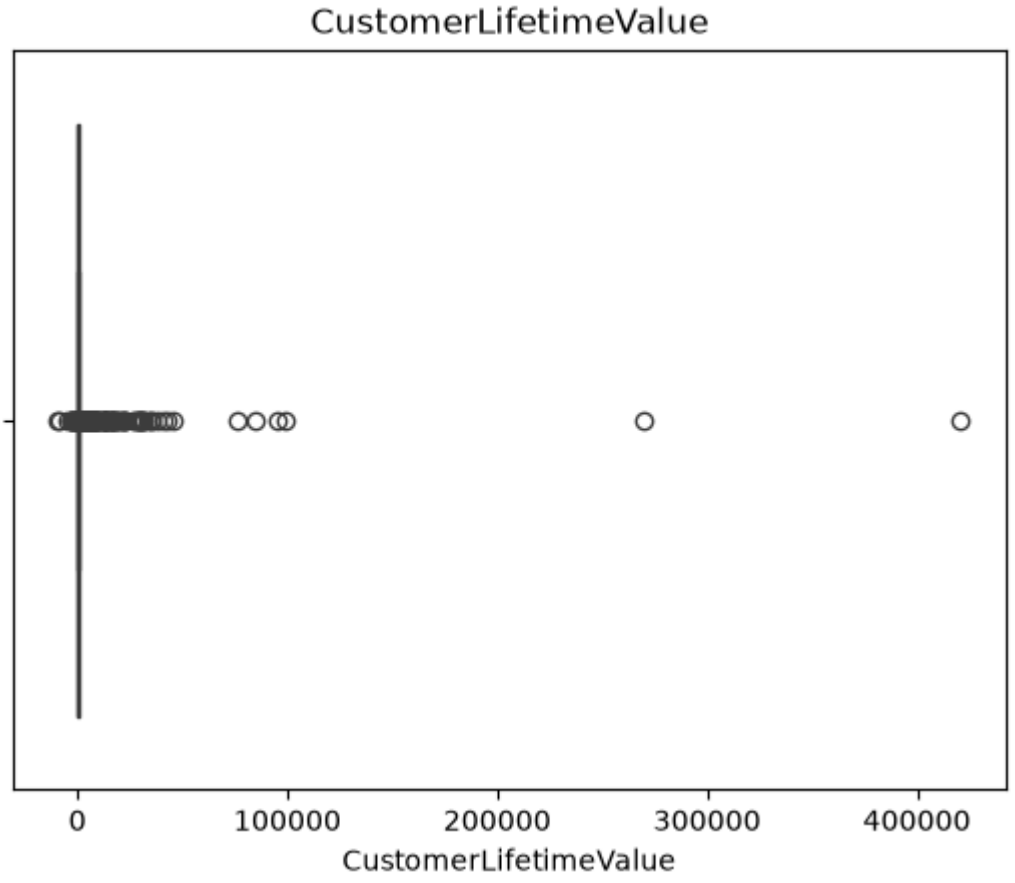
In [14]:
```
plt.figure(figsize = (10,5))

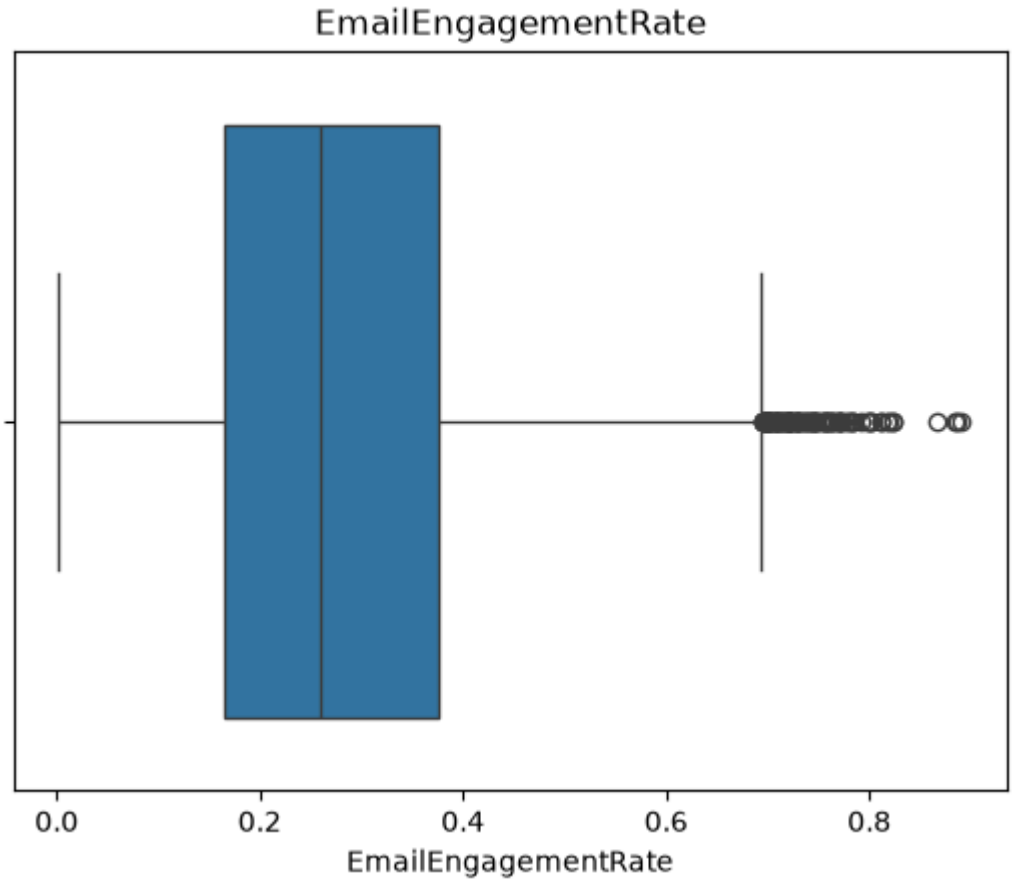
for i in num:
    sns.boxplot(data = df ,x = i)
    plt.title(i)
    plt.show()
```

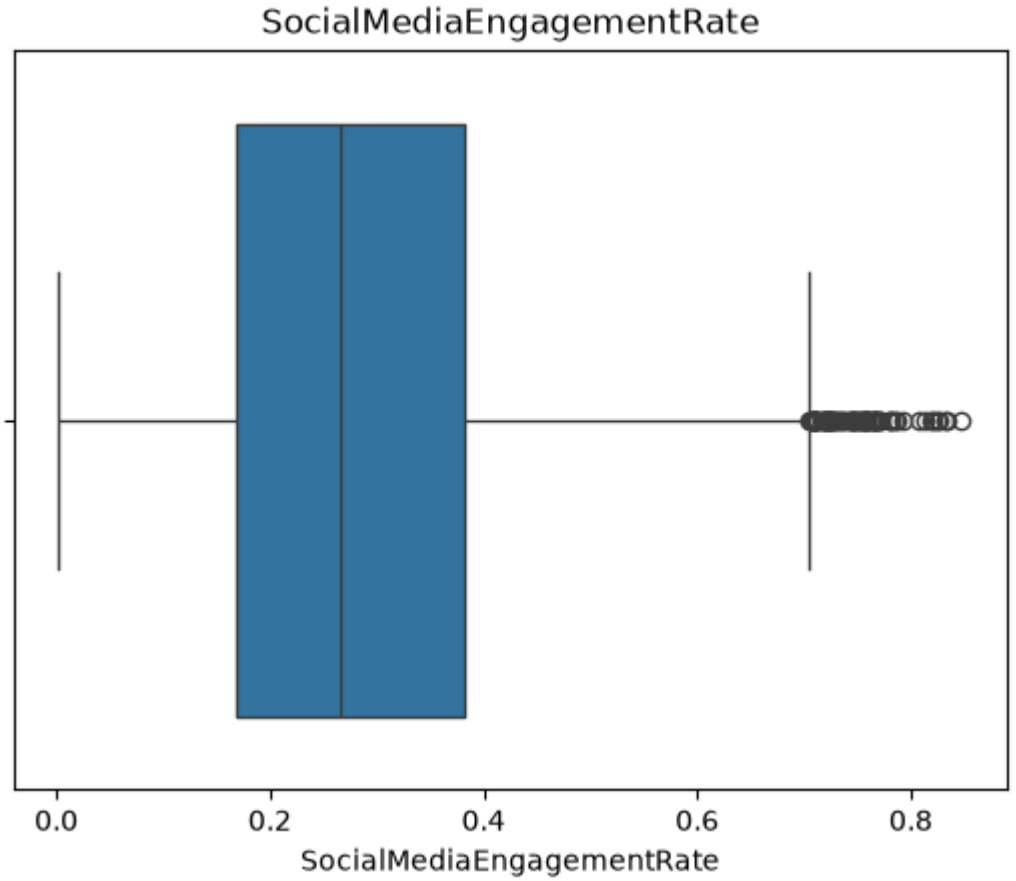


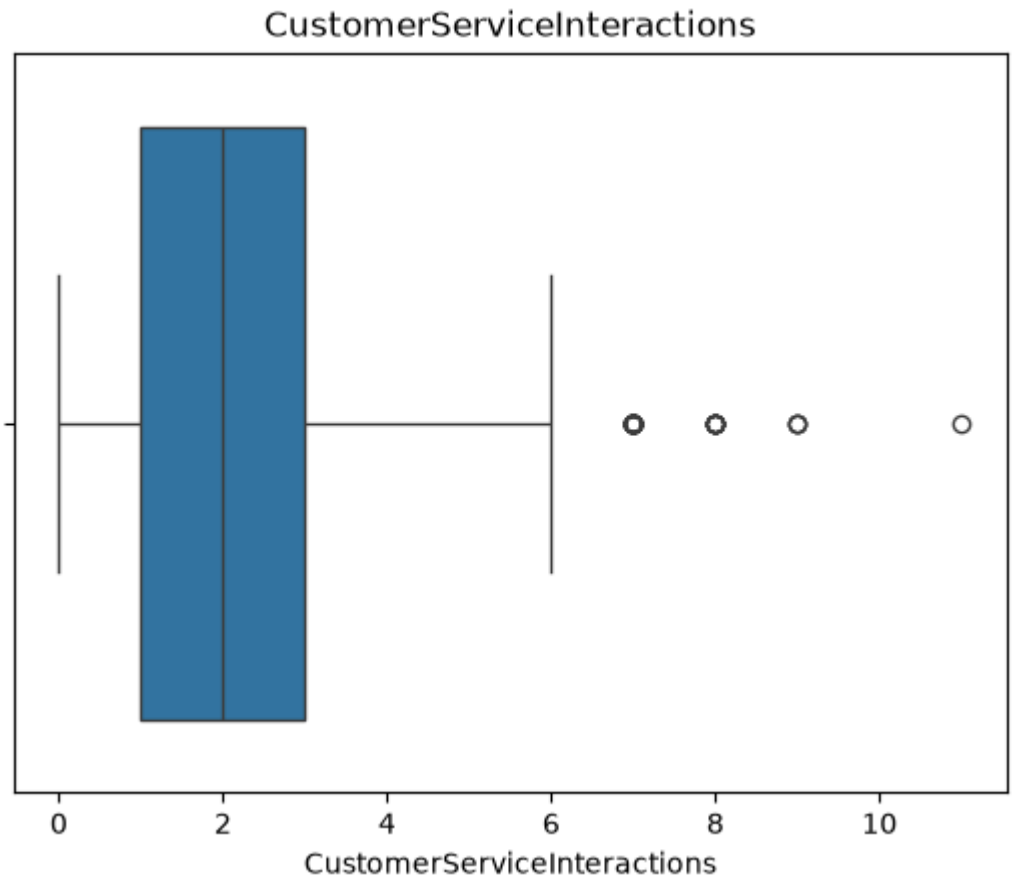


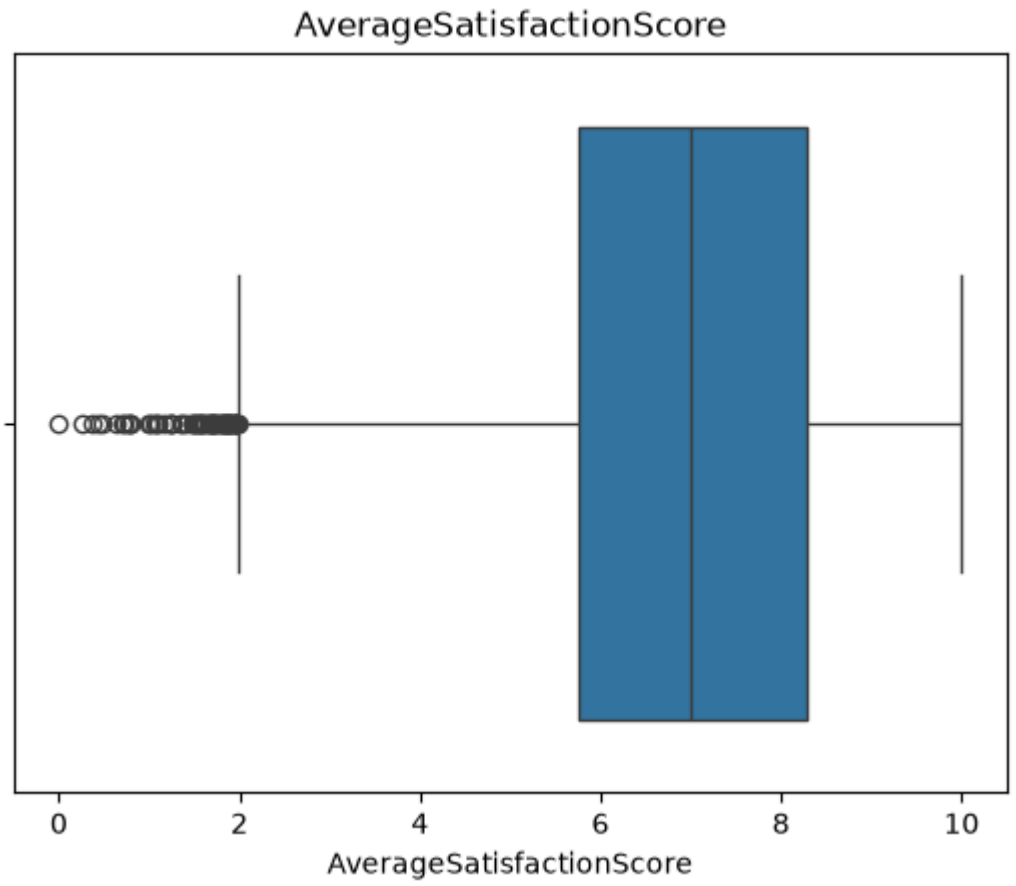

AverageOrderValue
0
10000
20000
30000
40000
50000
AverageOrderValue

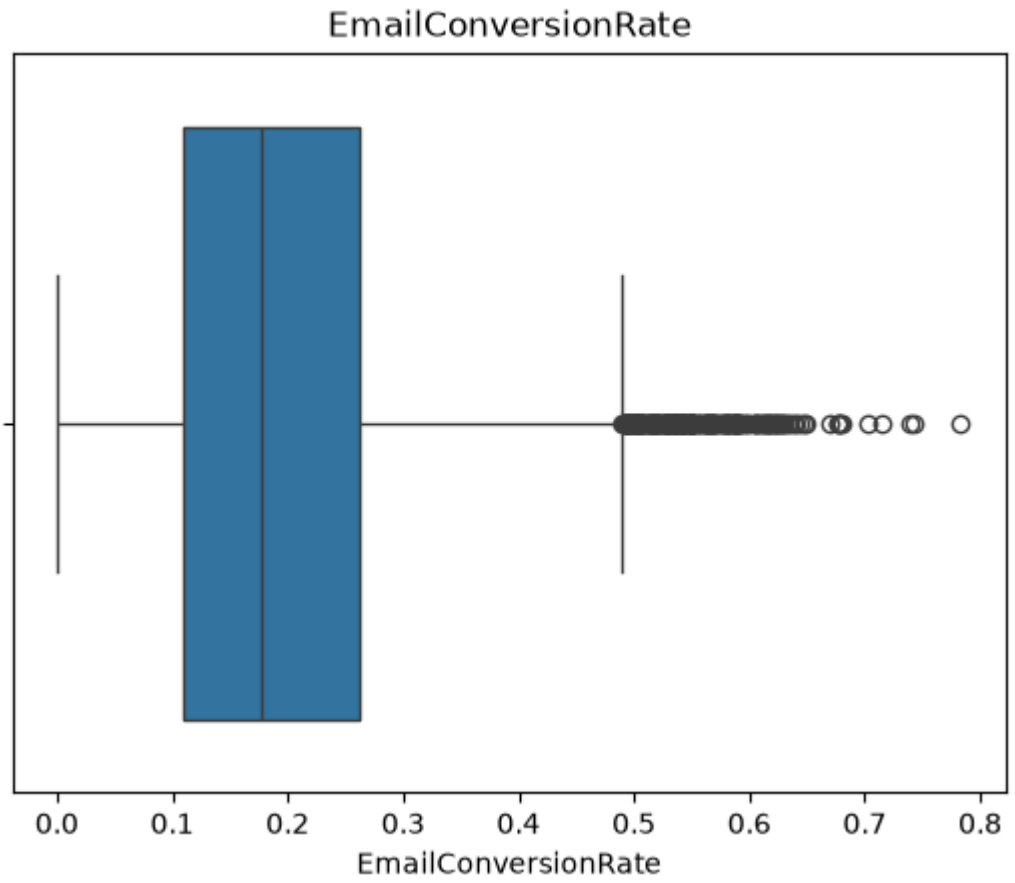

CustomerLifetimeValue
0
100000
200000
300000
400000
CustomerLifetimeValue

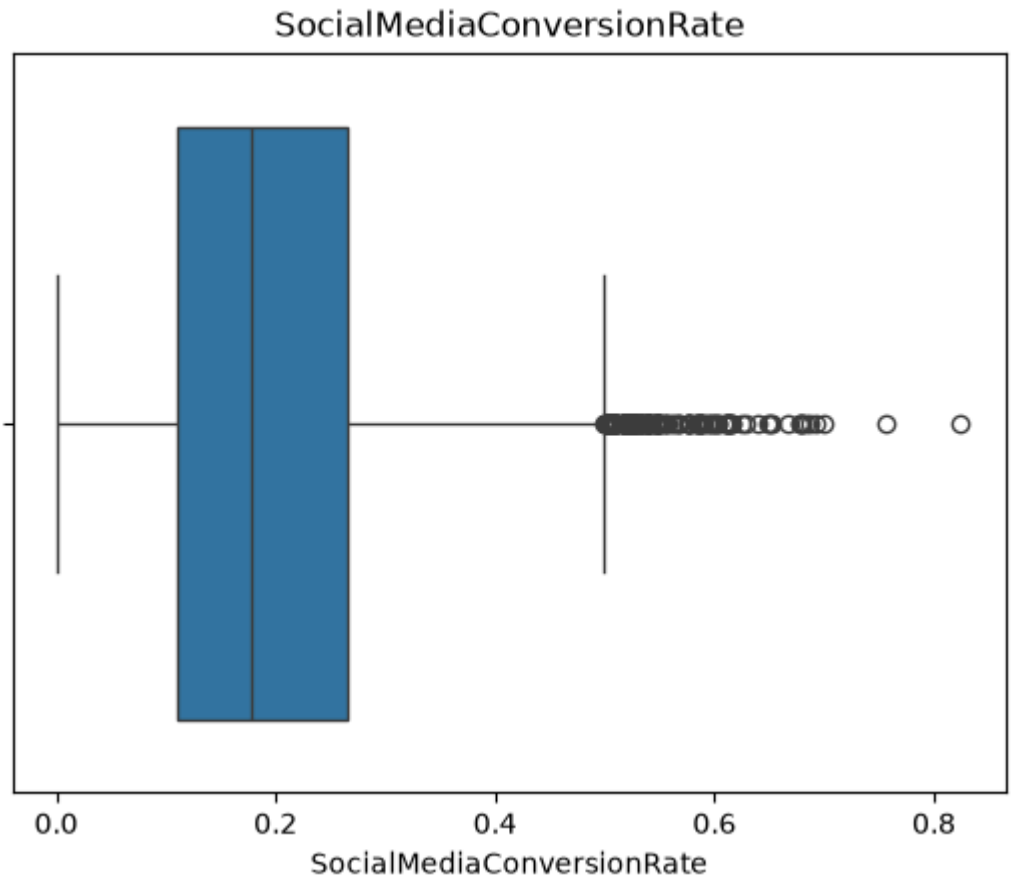

EmailEngagementRate
0.0
0.2
0.4
0.6
0.8
EmailEngagementRate

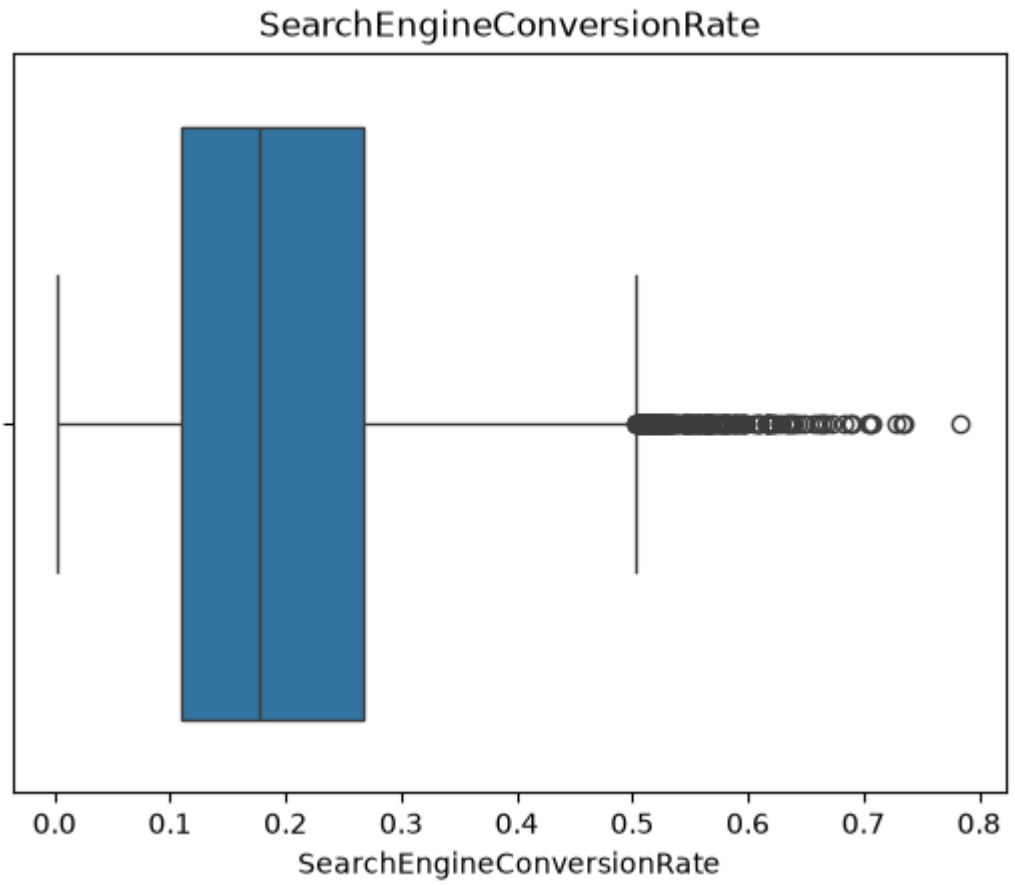

SocialMediaEngagementRate
0.0
0.2
0.4
0.6
0.8
SocialMediaEngagementRate


CustomerServiceInteractions
0
2
4
6
8
10
CustomerServiceInteractions


AverageSatisfactionScore
0
2
4
6
8
10
AverageSatisfactionScore


EmailConversionRate
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
EmailConversionRate


SocialMediaConversionRate
0.0
0.2
0.4
0.6
0.8
SocialMediaConversionRate



In [15]:
```python
def outlier(df,col_name):
    q1  = df[i].quantile(0.25)
    q3 = df[i].quantile(0.75)

    iqr = q3 - q1

    lb = q1 - 1.5 * iqr
    ub = q3 + 1.5 * iqr

    return df[(df[i] < lb) | (df[i] > ub)]
```

In [16]:
```python
for i in num:
    print(i)
    print(len(outlier(df,i)))
```

```
Age
105
TotalPurchases
156
AverageOrderValue
915
CustomerLifetimeValue
935
EmailEngagementRate
107
SocialMediaEngagementRate
79
CustomerServiceInteractions
54
AverageSatisfactionScore
63
EmailConversionRate
204
SocialMediaConversionRate
171
SearchEngineConversionRate
186
```

5.treatment of outlier

In [17]:
```
from feature_engine.outliers import Winsorizer
```

In [18]:
```
num = df.select_dtypes(include="number").columns.tolist()
win_obj = Winsorizer(capping_method="iqr",tail = "both")
df = win_obj.fit_transform(df)
df
```

Out[18]:

	CustomerID	RegistrationDate	Age	Gender	IncomeLevel	Country	City
0	CUST00002	2020-05-10	25.0	Other	Very High	Other	Tokyo
1	CUST00002	2021-07-18	35.0	Prefer not to say	High	Germany	London
2	CUST00003	2021-02-04	43.0	Other	High	France	London
3	CUST00004	2020-12-31	49.0	Female	High	Australia	Los Angeles
4	CUST00005	2022-06-27	29.0	Female	Very High	Australia	Tokyo
...	...	...	...	...	...	...	...
9995	CUST09996	2018-09-29	39.0	Female	High	Japan	New York
9996	CUST09997	2020-10-25	58.0	Prefer not to say	Low	France	Sydney
9997	CUST09998	2023-04-07	14.0	Other	Very High	USA	New York
9998	CUST09999	2021-05-27	29.0	Other	High	Canada	Berlin
9999	CUST10000	2021-02-15	32.0	Other	Low	France	Paris

10000 rows × 23 columns

In [19]:
```
for i in num:
    print(i)
    print(len(outlier(df,i)))
```

```
Age
0
TotalPurchases
0
AverageOrderValue
0
CustomerLifetimeValue
0
EmailEngagementRate
0
SocialMediaEngagementRate
0
CustomerServiceInteractions
0
AverageSatisfactionScore
0
EmailConversionRate
0
SocialMediaConversionRate
0
SearchEngineConversionRate
0
```

6. checking for duplicates

In [20]:
```
df.duplicated().sum()
```

Out[20]:
```
np.int64(0)
```

7.checking data inconsistent ot not

In [21]:
```
for i in num:
    print("\n",i)
    print(df[i].unique())
```

```
 Age
[25.  35.  43.  49.  29.  57.  23.  16.  64.  28.  53.  37.  40.  48.
 47.  42.  41.  38.  31.  30.  21.  72.5 56.  19.  45.   9.  44.   1.
 36.  -3.5  8.  22.  27.  13.  52.  46.  33.  24.  68.  54.  50.  20.
  0.  26.   6.  18.  55.  34.   2.  12.  59.  32.  14.  61.  10.  65.
 15.   4.  63.  39.  17.  51.  71.  58.   3.  60.  11.   5.  67.  69.
 -2.  62.   7.  66.  70.  72.  -1.  -3. ]

 TotalPurchases
[ 4.  6. 10.  5.  7.  3.  0.  1.  2.  9. 13.  8. 11. 12.]

 AverageOrderValue
[ 15.88650949  27.63885266 161.73942549 ...  37.19689899  13.19195151
 122.95839734]

 CustomerLifetimeValue
[ 248.01823776  181.72505558 1147.51690882 ...  117.20350868   -6.65524745
  431.47973223]

 EmailEngagementRate
[0.33236451 0.34457428 0.40965598 ... 0.4877044  0.37762681 0.27661177]

 SocialMediaEngagementRate
[0.37969395 0.14098787 0.32365974 ... 0.12601827 0.60011191 0.05209878]

 CustomerServiceInteractions
[0. 1. 2. 4. 6. 3. 5.]

 AverageSatisfactionScore
[6.24088137 7.7219169  7.89904167 ... 8.61034454 9.86424541 6.16303179]

 EmailConversionRate
[0.11295507 0.30097874 0.17663867 ... 0.35569396 0.22285612 0.20152959]

 SocialMediaConversionRate
[0.23994767 0.23082104 0.1322389  ... 0.10144462 0.16071648 0.30750717]

 SearchEngineConversionRate
[0.15677576 0.29073477 0.05050547 ... 0.16146929 0.43102282 0.04113157]
```

In [22]:
```
for j in cat:
    print("\n",j)
    print(df[j].unique())
```

```
 CustomerID
<StringArray>
['CUST00002', 'CUST00003', 'CUST00004', 'CUST00005', 'CUST00007', 'CUST000
08',
 'CUST00009', 'CUST00010', 'CUST00011', 'CUST00012',
 ...
 'CUST09991', 'CUST09992', 'CUST09993', 'CUST09994', 'CUST09995', 'CUST099
96',
 'CUST09997', 'CUST09998', 'CUST09999', 'CUST10000']
Length: 9508, dtype: str

 RegistrationDate
<StringArray>
['2020-05-10', '2021-07-18', '2021-02-04', '2020-12-31', '2022-06-27',
 '2023-12-10', '2019-04-12', '2021-05-23', '2018-11-27', '2022-01-22',
 ...
 '2023-05-09', '2018-09-21', '2018-11-18', '2019-11-08', '2023-05-20',
 '2019-06-28', '2022-02-21', '2018-02-03', '2021-03-21', '2019-04-11']
Length: 2159, dtype: str

 Gender
<StringArray>
['Other', 'Prefer not to say', 'Female', 'Male', 'M', 'F']
Length: 6, dtype: str

 IncomeLevel
<StringArray>
['Very High', 'High', 'Low', 'Medium', 'L', 'H']
Length: 6, dtype: str

 Country
<StringArray>
['Other', 'Germany', 'France', 'Australia', 'USA', 'Canada', 'UK', 'Japa
n']
Length: 8, dtype: str

 City
<StringArray>
[      'Tokyo',      'London', 'Los Angeles',      'Sydney',      'Berli
n',
    'New York',       'Paris',       'Other']
Length: 8, dtype: str

 FavoriteCategory
<StringArray>
[   'Clothing', 'Electronics',      'Beauty',      'Sports',  'Home Good
s',
        'Food',        'Toys',       'Books']
Length: 8, dtype: str

 SecondFavoriteCategory
<StringArray>
[      'Books',        'Toys',    'Clothing',      'Beauty',  'Home Good
s',
      'Sports',        'Food', 'Electronics']
Length: 8, dtype: str

 MobileAppUsage
<StringArray>
['High', 'Low', 'Never', 'Medium']
```

```
Length: 4, dtype: str

 RepeatCustomer
<StringArray>
['Yes', 'No']
Length: 2, dtype: str

 PremiumMember
<StringArray>
['Yes', 'No']
Length: 2, dtype: str

 HasReturnedItems
<StringArray>
['No', 'Yes']
Length: 2, dtype: str
```

In [23]:
```
df["Gender"] = df["Gender"].replace({"M":"Male","F":"Female"})
```

In [24]:
```
df["IncomeLevel"] = df["IncomeLevel"].replace({"L":"Low","H":"High"})
```

In [25]:
```
for j in cat:
    print("\n",j)
    print(df[j].unique())
```

```
 CustomerID
<StringArray>
['CUST00002', 'CUST00003', 'CUST00004', 'CUST00005', 'CUST00007', 'CUST000
08',
 'CUST00009', 'CUST00010', 'CUST00011', 'CUST00012',
 ...
 'CUST09991', 'CUST09992', 'CUST09993', 'CUST09994', 'CUST09995', 'CUST099
96',
 'CUST09997', 'CUST09998', 'CUST09999', 'CUST10000']
Length: 9508, dtype: str

 RegistrationDate
<StringArray>
['2020-05-10', '2021-07-18', '2021-02-04', '2020-12-31', '2022-06-27',
 '2023-12-10', '2019-04-12', '2021-05-23', '2018-11-27', '2022-01-22',
 ...
 '2023-05-09', '2018-09-21', '2018-11-18', '2019-11-08', '2023-05-20',
 '2019-06-28', '2022-02-21', '2018-02-03', '2021-03-21', '2019-04-11']
Length: 2159, dtype: str

 Gender
<StringArray>
['Other', 'Prefer not to say', 'Female', 'Male']
Length: 4, dtype: str

 IncomeLevel
<StringArray>
['Very High', 'High', 'Low', 'Medium']
Length: 4, dtype: str

 Country
<StringArray>
['Other', 'Germany', 'France', 'Australia', 'USA', 'Canada', 'UK', 'Japa
n']
Length: 8, dtype: str

 City
<StringArray>
[      'Tokyo',      'London', 'Los Angeles',      'Sydney',      'Berli
n',
    'New York',       'Paris',       'Other']
Length: 8, dtype: str

 FavoriteCategory
<StringArray>
[   'Clothing', 'Electronics',      'Beauty',      'Sports',  'Home Good
s',
        'Food',        'Toys',       'Books']
Length: 8, dtype: str

 SecondFavoriteCategory
<StringArray>
[      'Books',        'Toys',    'Clothing',      'Beauty',  'Home Good
s',
      'Sports',        'Food', 'Electronics']
Length: 8, dtype: str

 MobileAppUsage
<StringArray>
['High', 'Low', 'Never', 'Medium']
```

```
Length: 4, dtype: str

 RepeatCustomer
<StringArray>
['Yes', 'No']
Length: 2, dtype: str

 PremiumMember
<StringArray>
['Yes', 'No']
Length: 2, dtype: str

 HasReturnedItems
<StringArray>
['No', 'Yes']
Length: 2, dtype: str
```

8. Data Manipulation

8.1 Question:

- Get the Age and Gender of the first 10 customers.
- Get customers whose age is greater than 40

In [26]:
```
df.loc[0:9, ["Age", "Gender"]]
```

Out[26]:

	Age	Gender
0	25.0	Other
1	35.0	Prefer not to say
2	43.0	Other
3	49.0	Female
4	29.0	Female
5	57.0	Male
6	23.0	Other
7	57.0	Other
8	43.0	Prefer not to say
9	16.0	Other

In [27]:
```
df.loc[df["Age"] > 40, ["CustomerID", "Age", "Gender"]]
```

Out[27]:

	CustomerID	Age	Gender
2	CUST00003	43.0	Other
3	CUST00004	49.0	Female
5	CUST00002	57.0	Male
7	CUST00008	57.0	Other
8	CUST00009	43.0	Prefer not to say
...	...	...	...
9967	CUST09968	58.0	Other
9979	CUST09980	53.0	Other
9982	CUST09983	43.0	Other
9986	CUST09987	46.0	Female
9996	CUST09997	58.0	Prefer not to say

3300 rows × 3 columns

8.2 performing the loc, iloc, at, iat, query, fancy indexing for above data

In [28]:
```
df.iloc[0:5, 0:4]
```

Out[28]:

	CustomerID	RegistrationDate	Age	Gender
0	CUST00002	2020-05-10	25.0	Other
1	CUST00002	2021-07-18	35.0	Prefer not to say
2	CUST00003	2021-02-04	43.0	Other
3	CUST00004	2020-12-31	49.0	Female
4	CUST00005	2022-06-27	29.0	Female

In [29]:
```
df.loc[0:9, ["Age", "Gender"]]
```

Out[29]:

	Age	Gender
0	25.0	Other
1	35.0	Prefer not to say
2	43.0	Other
3	49.0	Female
4	29.0	Female
5	57.0	Male
6	23.0	Other
7	57.0	Other
8	43.0	Prefer not to say
9	16.0	Other

In [30]:
```
df.at[0, "Age"]
```

Out[30]:
```
np.float64(25.0)
```

In [ ]:
```
df.iat[0, 2]
```

8.3query()

- Customers with Age > 40
- Customers with TotalPurchases > 10
- Customers older than 40 and having more than 5 purchases:

In [ ]:
```
df.query("Age > 40")
```

In [ ]:
```
df.query("TotalPurchases > 10")
```

In [ ]:
```
df.query("Age > 40 and TotalPurchases > 5")
```

9.Univariate Analysis

- Univariate = analyzing one column at a time.

9.1 Identify Numerical Columns

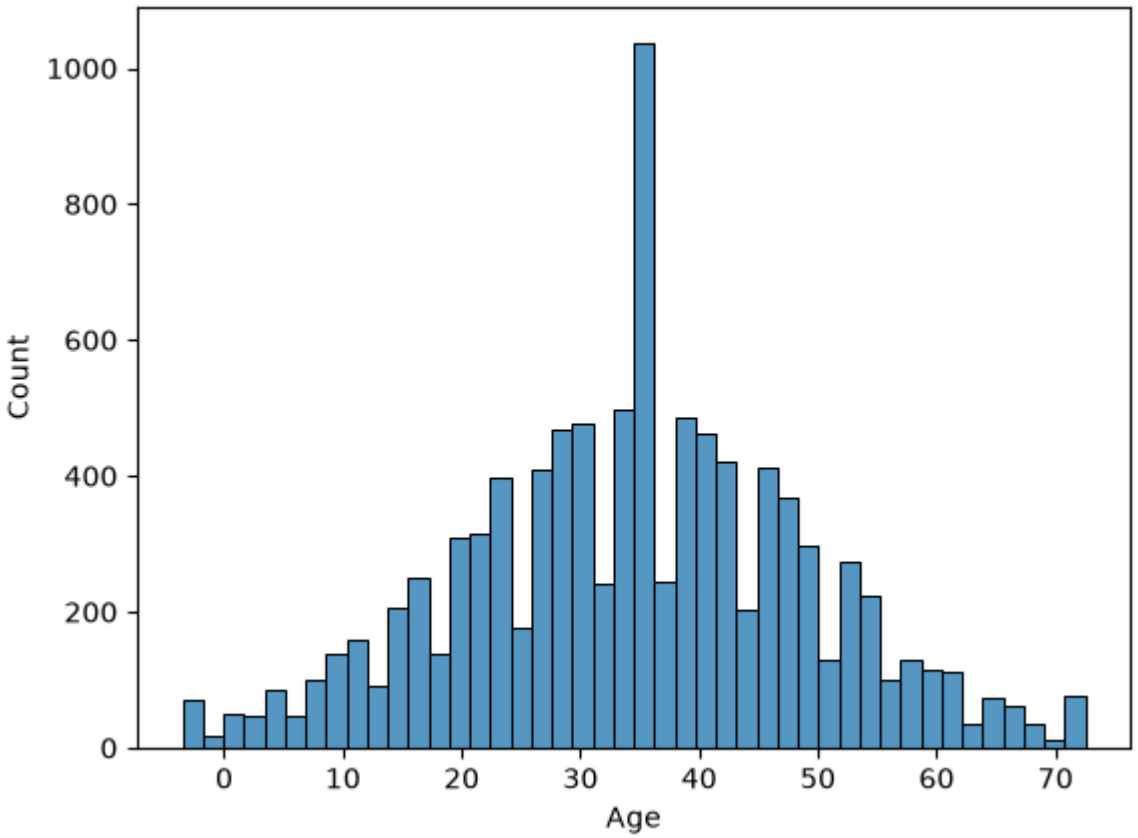
- use Histogram, KDE, Boxplot

In [ ]:
```
num = df.select_dtypes(include="number").columns
print(num)
```

9.1.1 Histogram

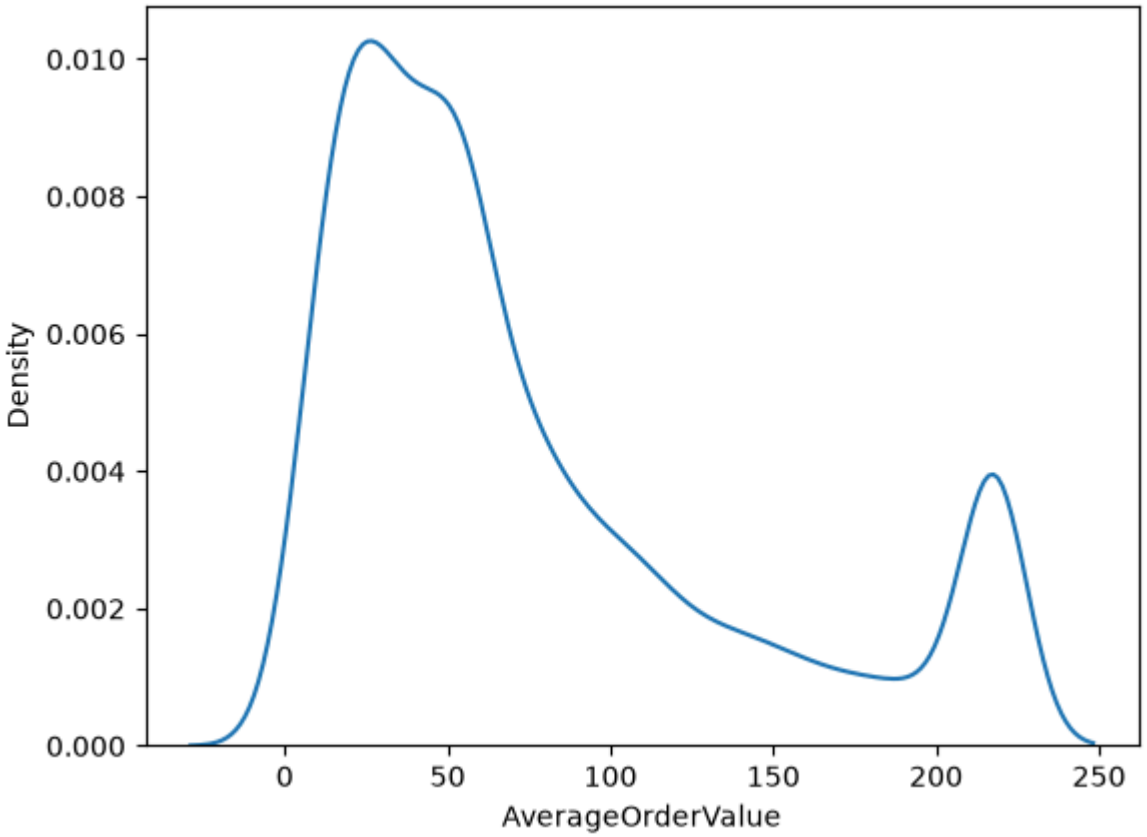
In [31]:
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(data=df, x="Age")
plt.show()
```



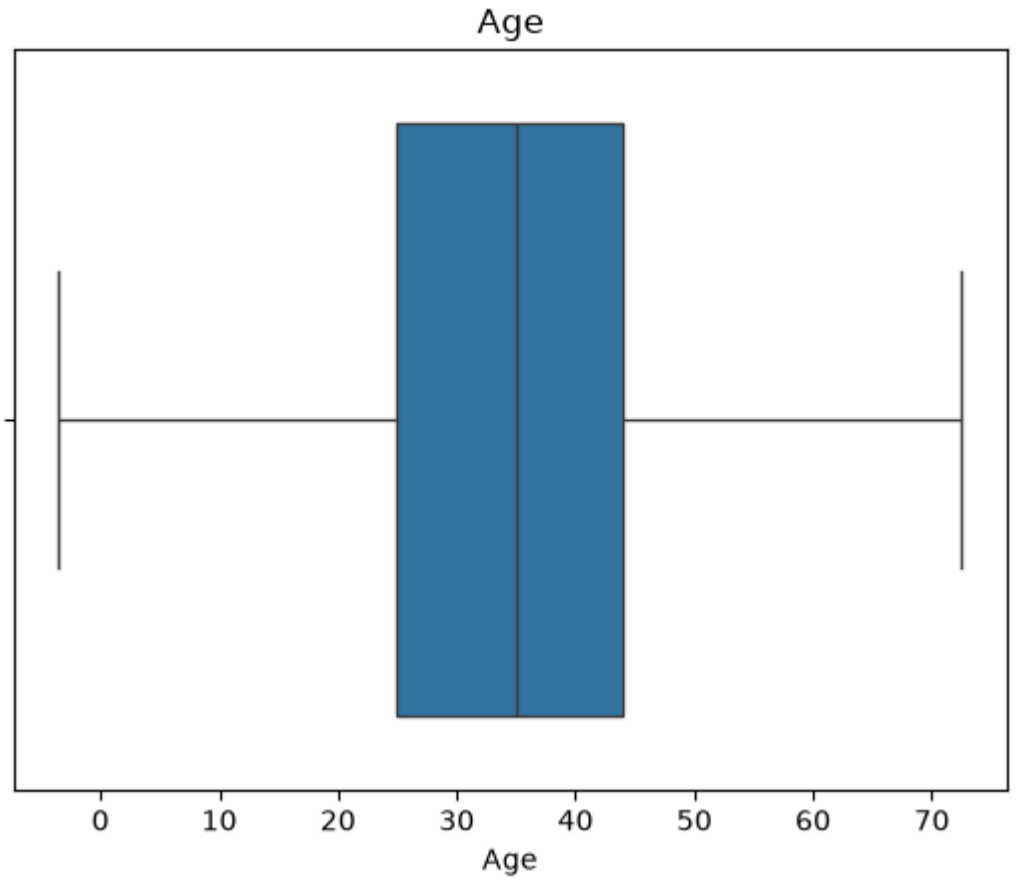
9.1.2 KDE plot

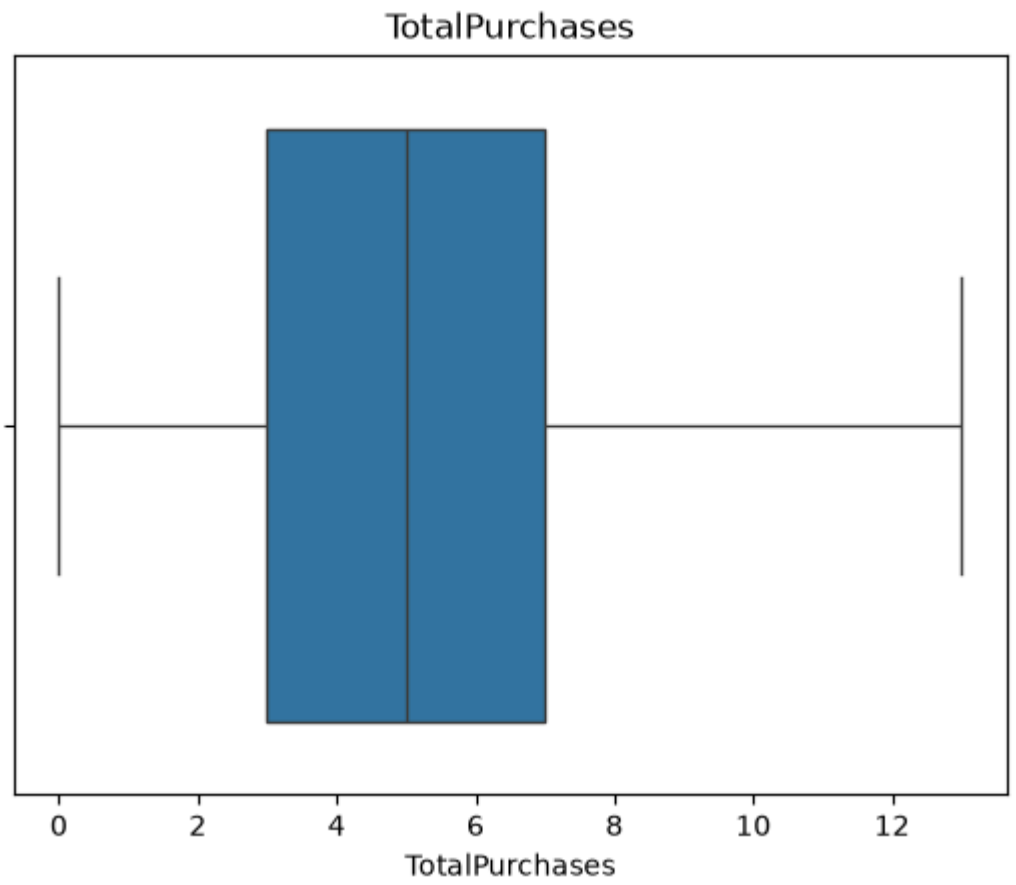
In [55]:
```
sns.kdeplot(data=df, x="AverageOrderValue")
plt.show()
```

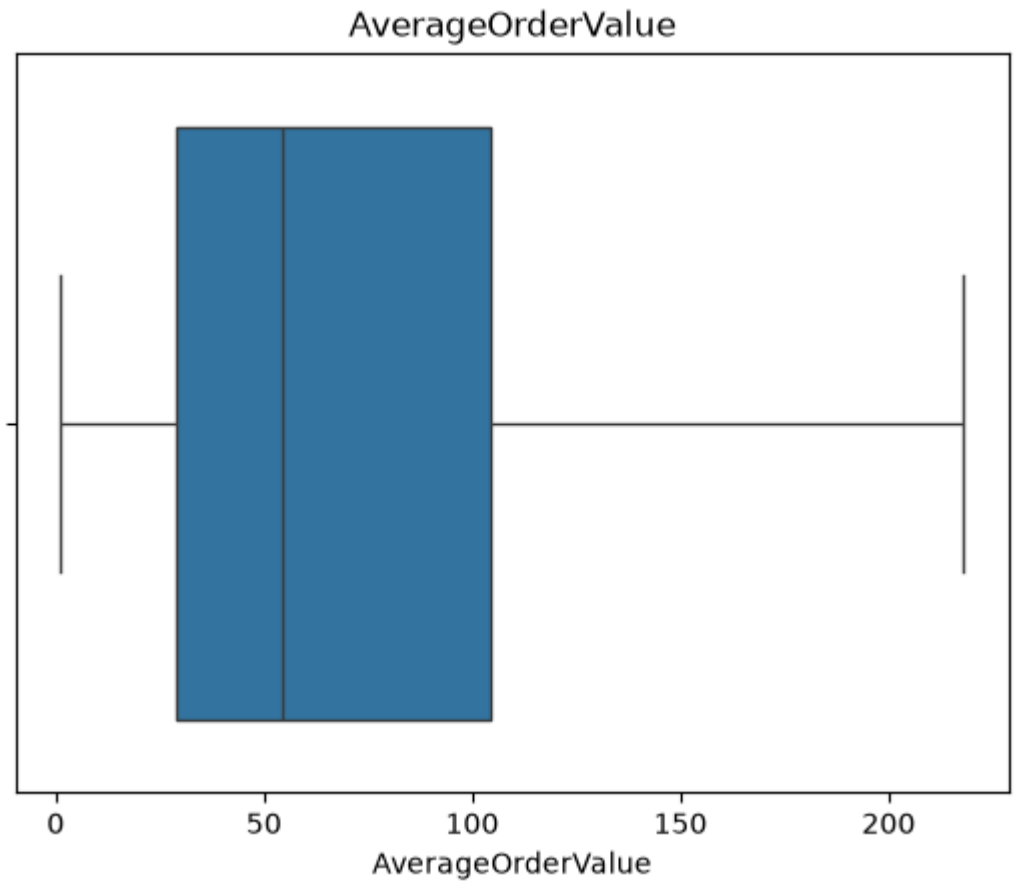


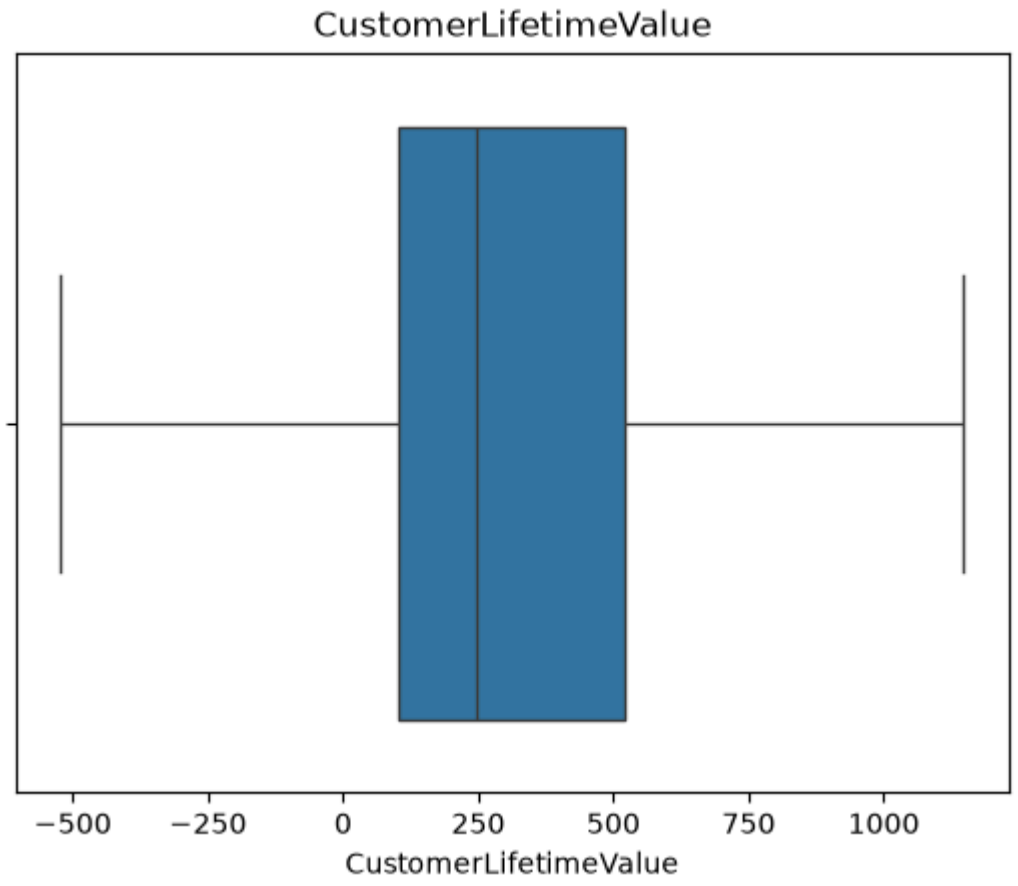
9.1.3 Boxplot

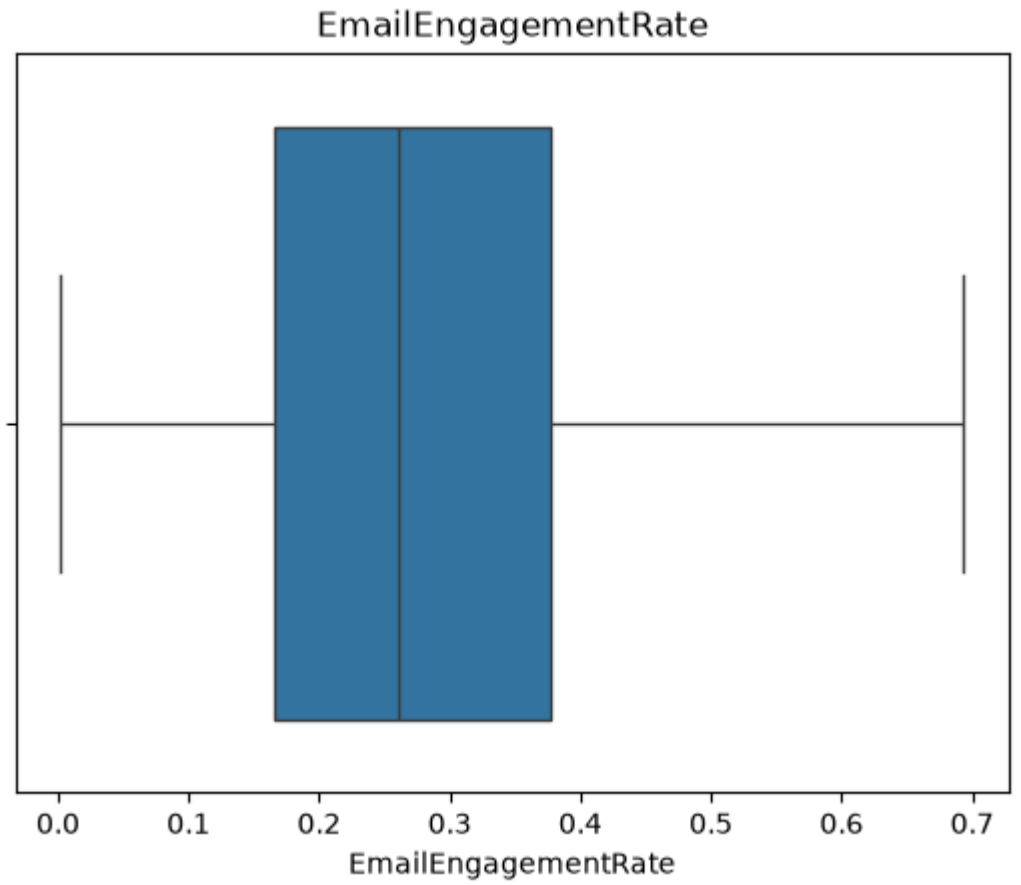
In [56]:
```
for i in num:
    sns.boxplot(data=df, x=i)
    plt.title(i)
    plt.show()
```

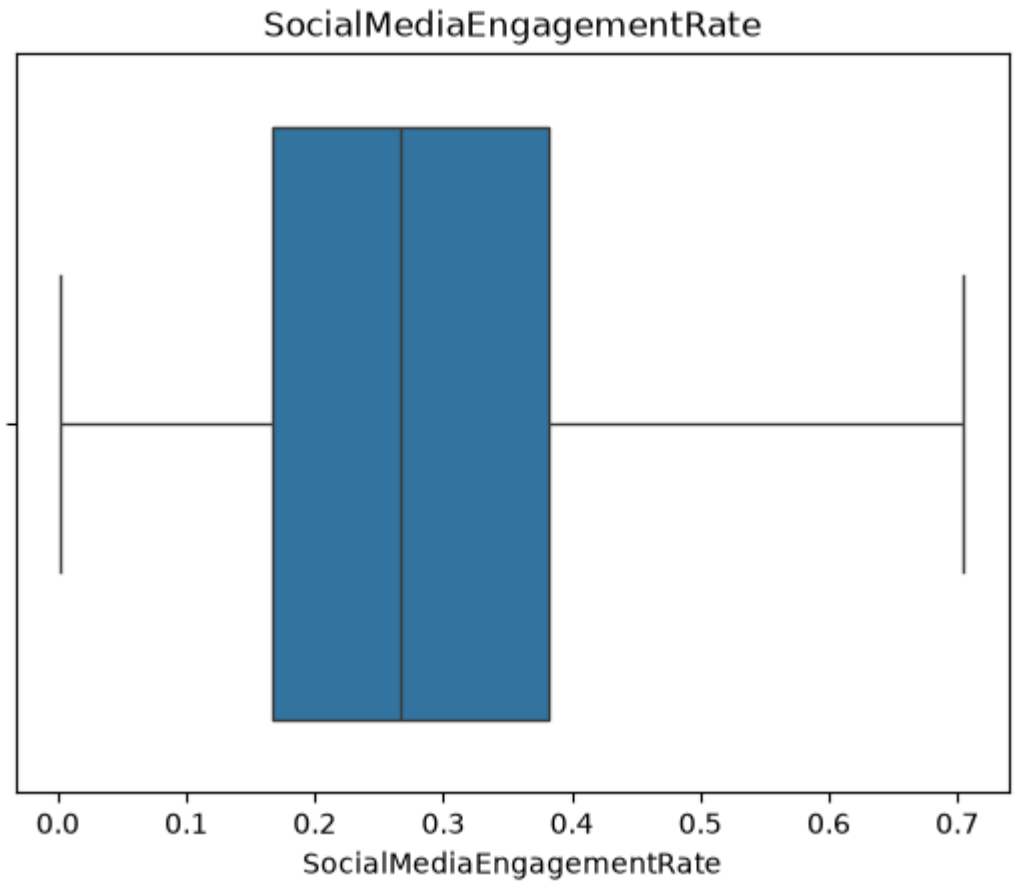

Age
0
10
20
30
40
50
60
70
Age

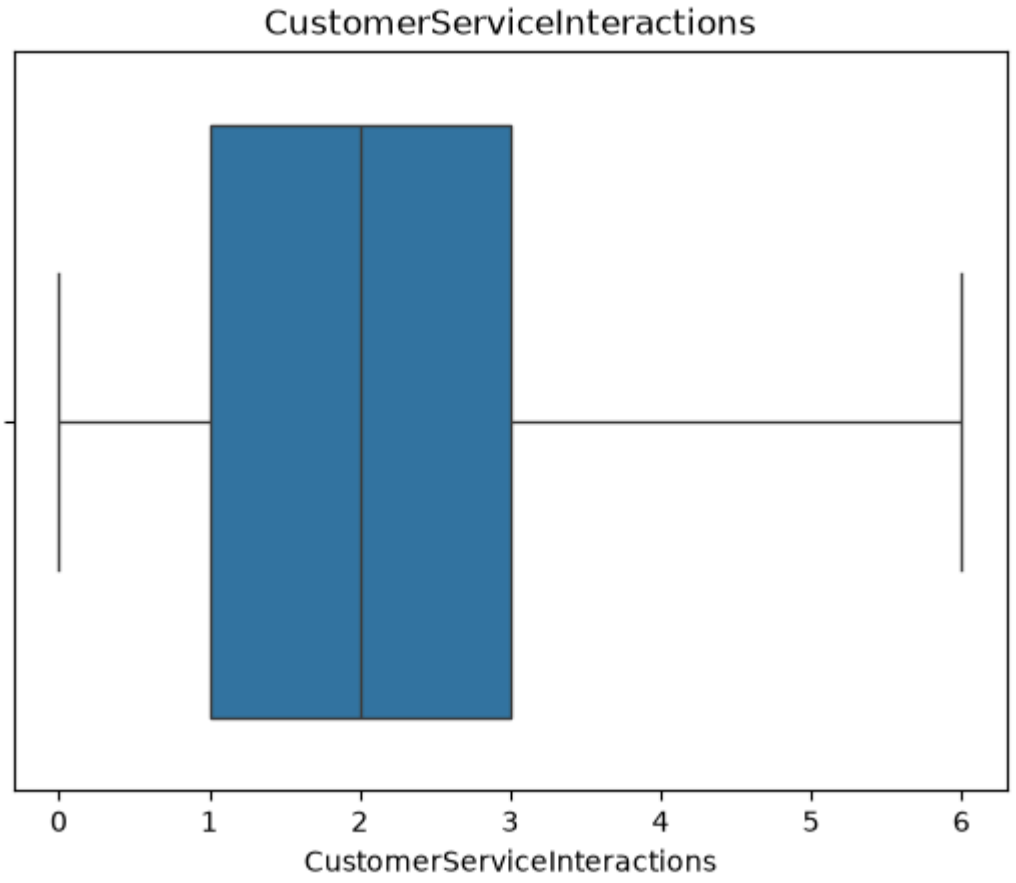

TotalPurchases
0
2
4
6
8
10
12
TotalPurchases

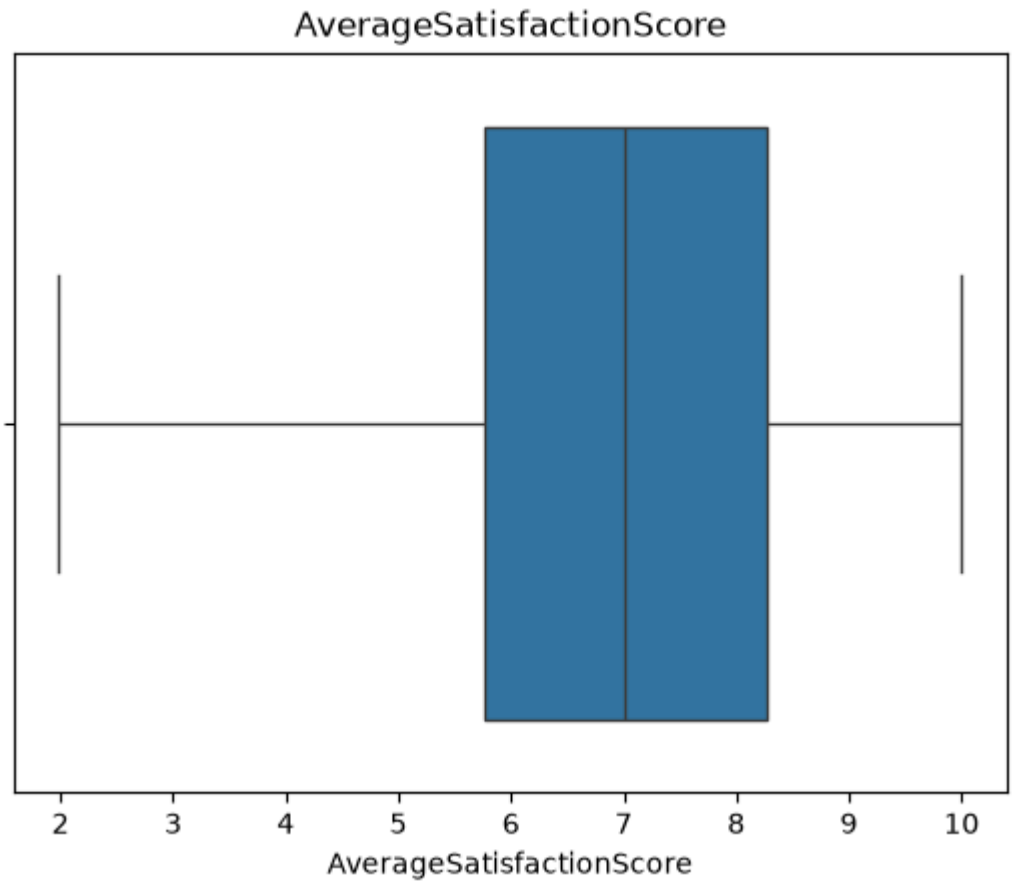

AverageOrderValue
0
50
100
150
200
AverageOrderValue

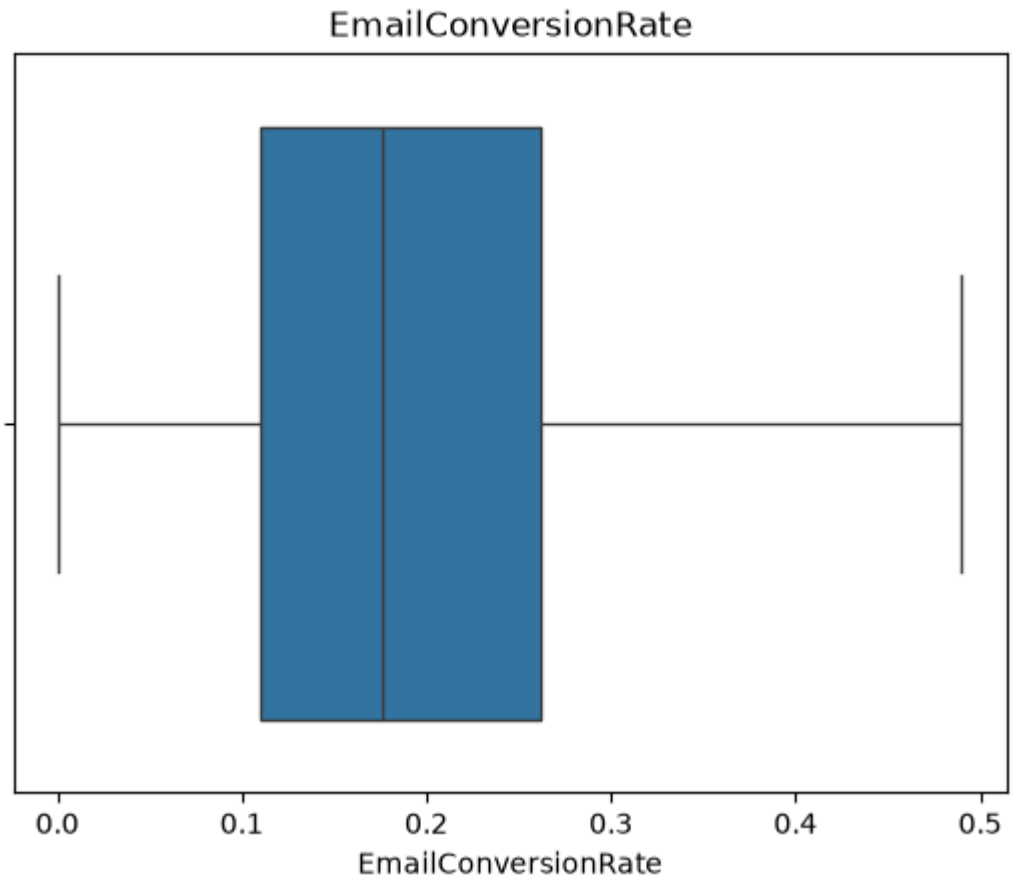

CustomerLifetimeValue
−500
−250
0
250
500
750
1000
CustomerLifetimeValue

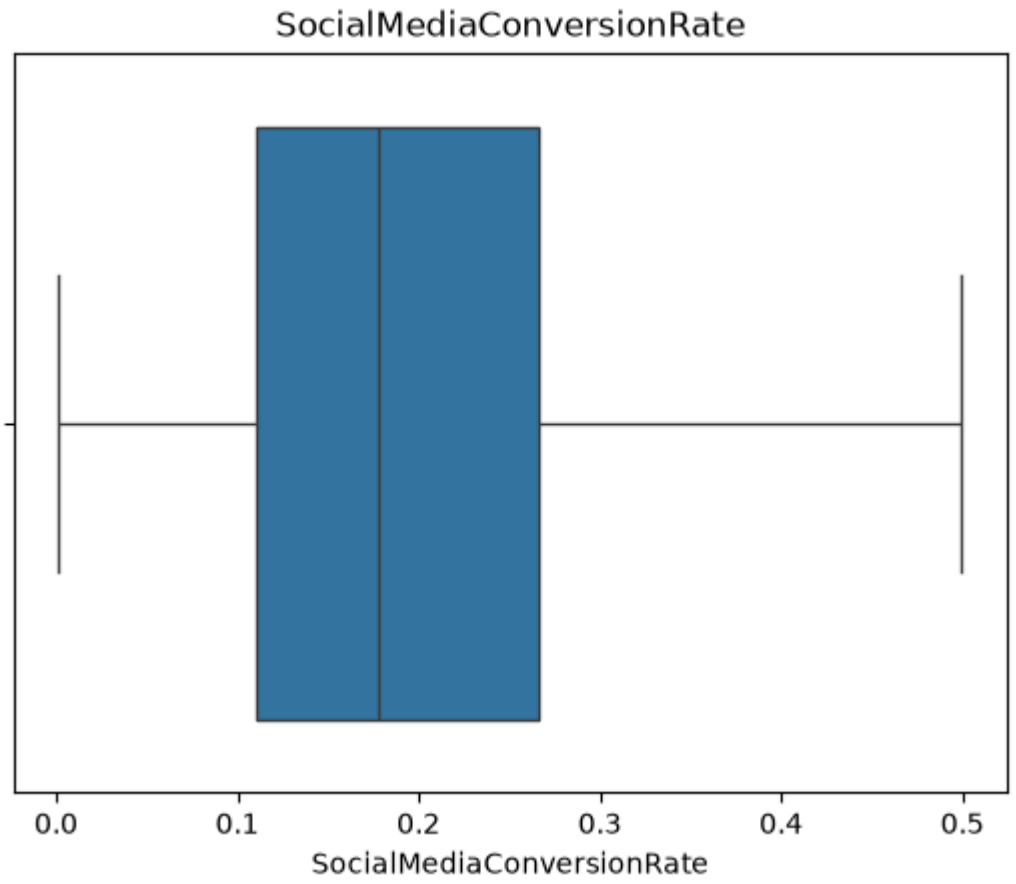

EmailEngagementRate
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
EmailEngagementRate

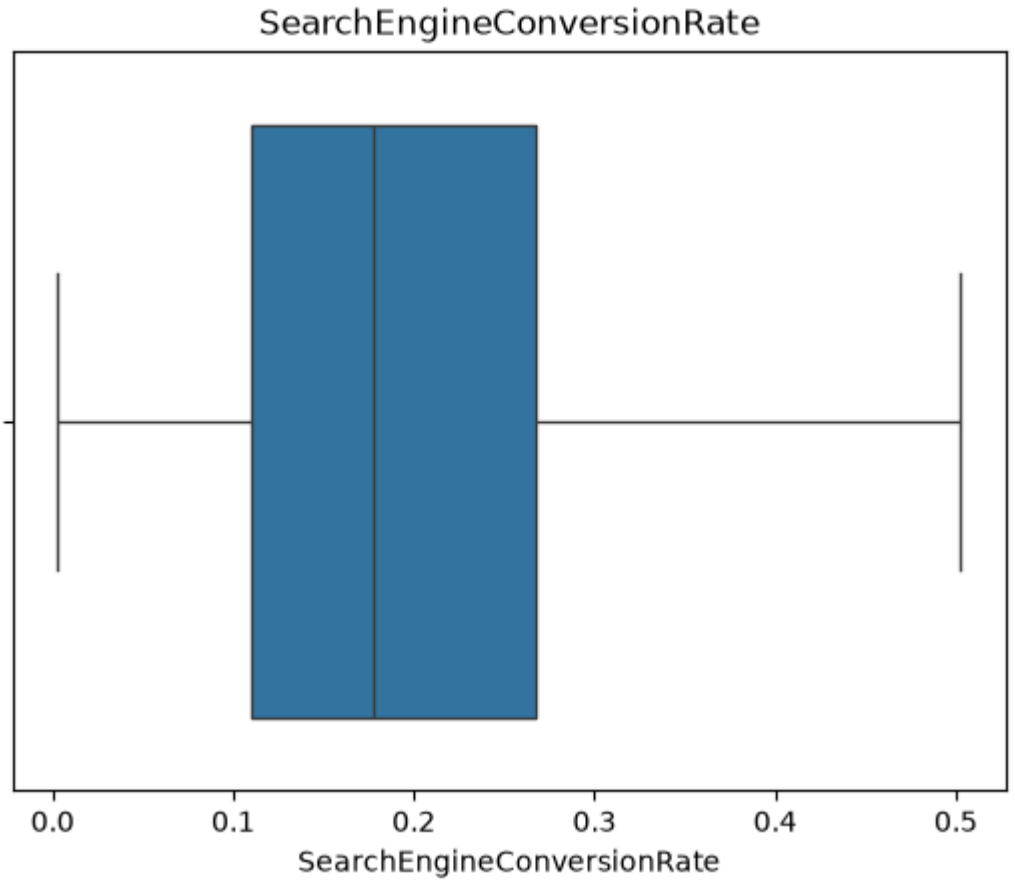

SocialMediaEngagementRate
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
SocialMediaEngagementRate


CustomerServiceInteractions
0
1
2
3
4
5
6
CustomerServiceInteractions


AverageSatisfactionScore
2
3
4
5
6
7
8
9
10
AverageSatisfactionScore


EmailConversionRate
0.0
0.1
0.2
0.3
0.4
0.5
EmailConversionRate


SocialMediaConversionRate
0.0
0.1
0.2
0.3
0.4
0.5
SocialMediaConversionRate

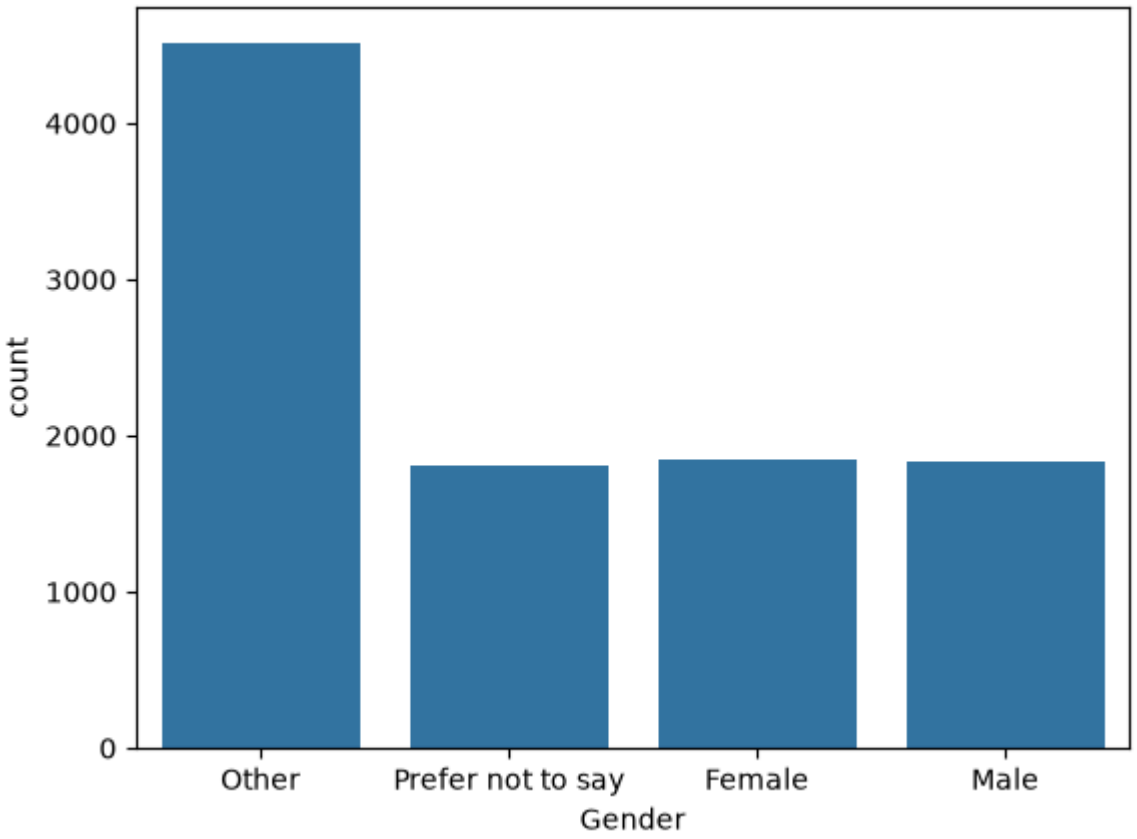


9.2 Categorical Columns

- for Categorical columns use - count , bar plot, pie chart

9.2.1 Countplot

In [64]:
```
sns.countplot(data=df, x="Gender")
plt.show()
```



9.2.2 Pie chart

In [ ]:
```
df["Gender"].value_counts().plot.pie(autopct="%1.1f%%")
plt.ylabel("")
plt.show()
```

10.Bivariate Analysis

10.1 Numerical × Numerical

- Here both variables are numerical.
- use Scatter plot

10.1.1 Scatter plot

In [ ]:
```
sns.scatterplot(
    data=df,
    x="Age",
    y="CustomerLifetimeValue"
)
plt.show()
```

10.2 Numerical × Categorical

use Box Plot, Bar Plot, Line Chart, Strip Plot, Swarm Plot, Violin Plot

10.2.1 Box Plot

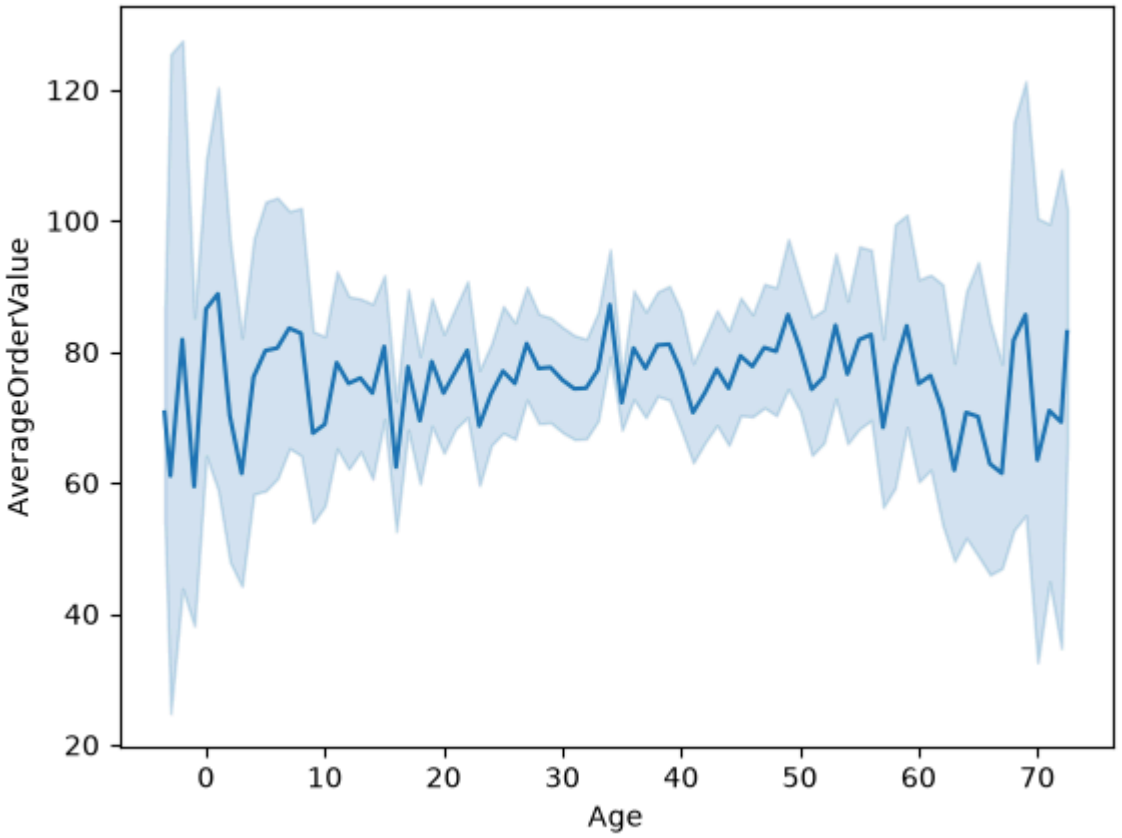
In [ ]:
```
sns.boxplot(
    data=df,
    x="Gender",
    y="CustomerLifetimeValue"
)
plt.show()
```

10.2.2 Bar Plot

In [ ]:
```
sns.barplot(
    data=df,
    x="Gender",
    y="CustomerLifetimeValue")
plt.show()
```

10.2.3 Line Chart

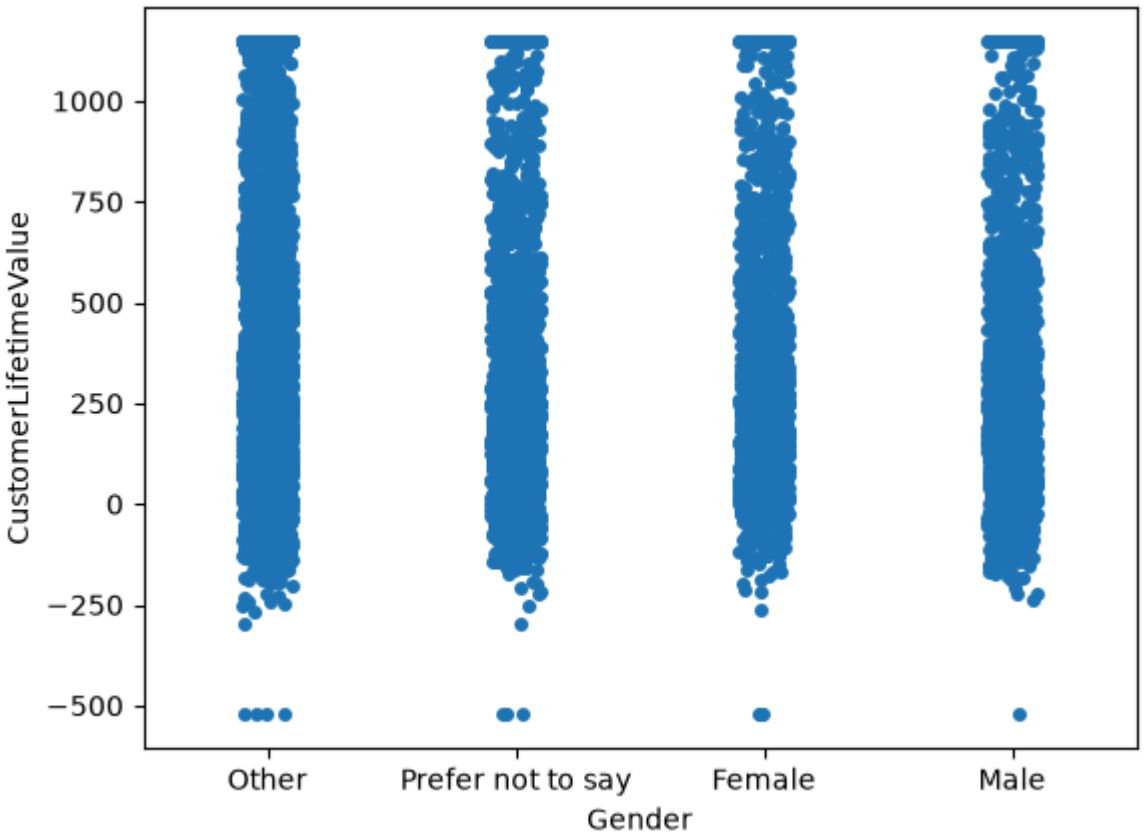
In [46]:
```
sns.lineplot(
    data=df,
    x="Age",
    y="AverageOrderValue"
)
plt.show()
```



10.2.4 Strip Plot

In [47]:
```
sns.stripplot(
    data=df,
    x="Gender",
    y="CustomerLifetimeValue"
```

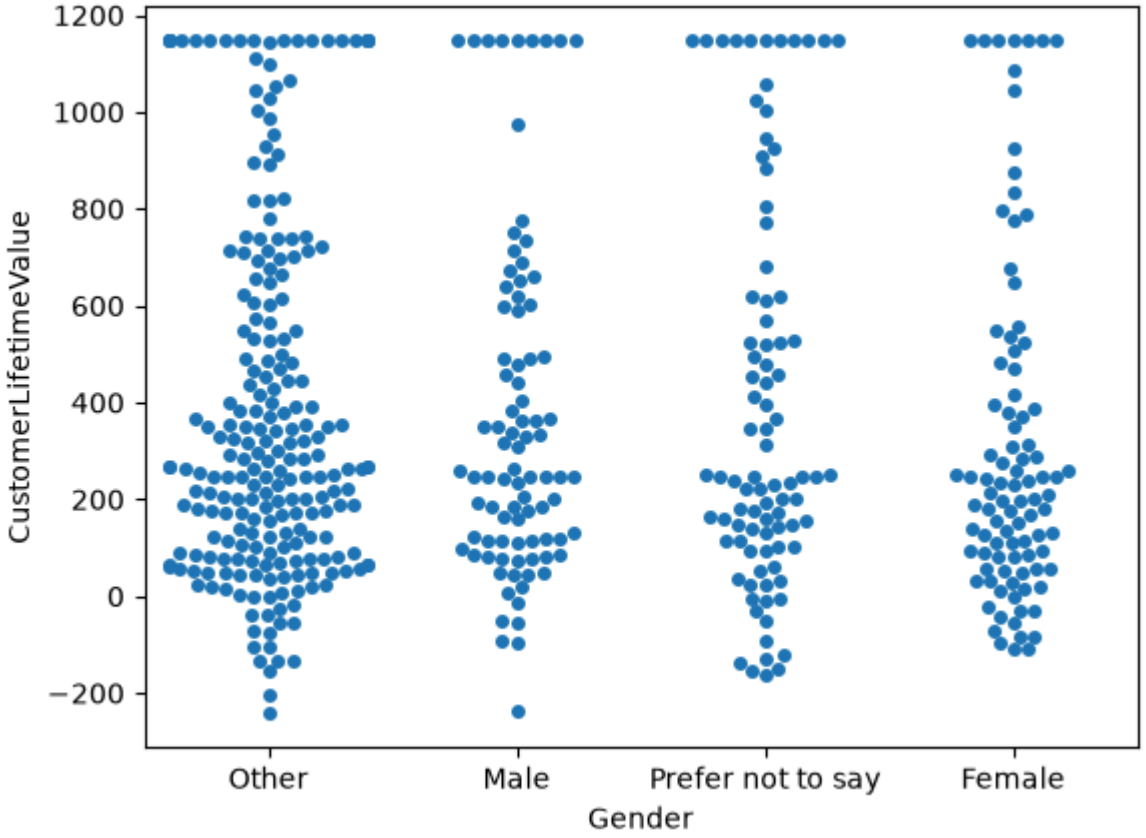
```
)
plt.show()
```



10.2.5 Swarm Plot

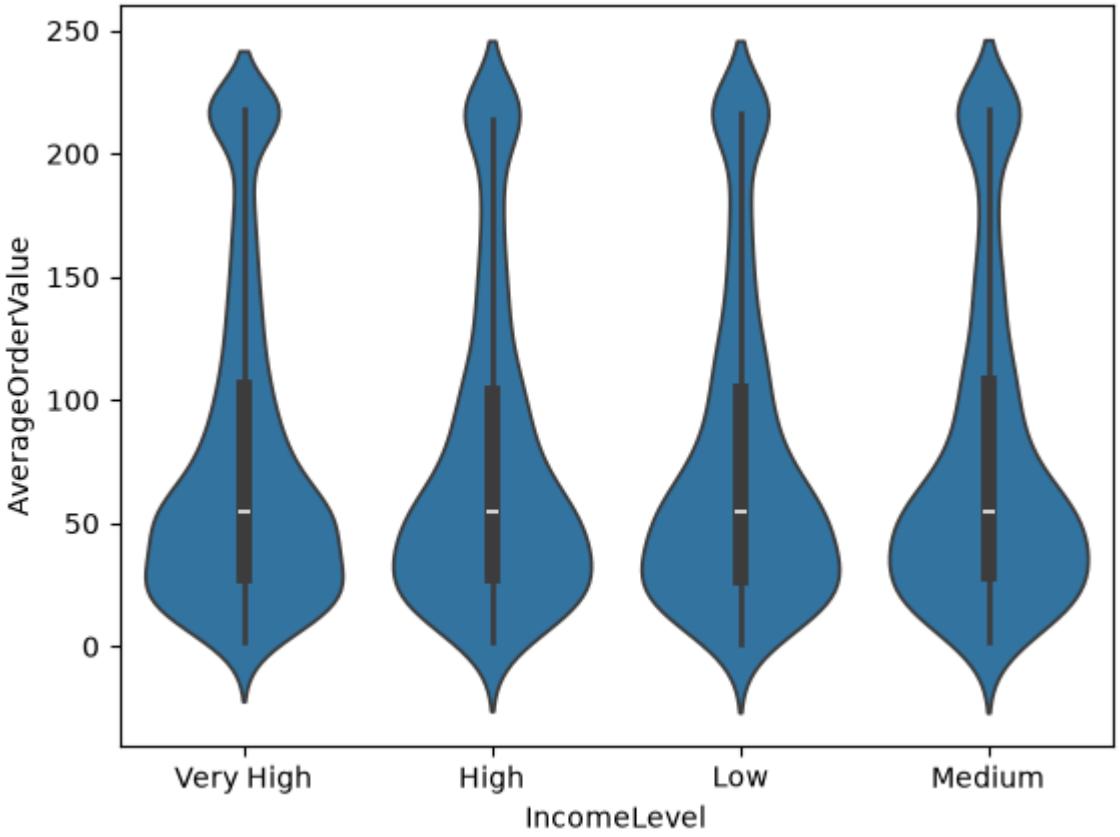
In [32]:
```
sns.swarmplot(
    data=df.sample(500),
    x="Gender",
    y="CustomerLifetimeValue"
)
plt.show()
```

```
C:\Users\mundr\anaconda3\envs\Mundru\Lib\site-packages\seaborn\categorical.py:3399: UserWarning: 6.6% of the points cannot be placed; you may want to decrease the size of the markers or use stripplot.
  warnings.warn(msg, UserWarning)
```



10.2.6 Violin Plot

In [33]:
```
sns.violinplot(
    data=df,
    x="IncomeLevel",
    y="AverageOrderValue"
)
plt.show()
```



10.3 Categorical × Categorical

- Both columns are categorical.
- use crosstab, bar plot

10.3.1 Crosstab

In [37]:
```
pd.crosstab(
    df["Gender"],
    df["PremiumMember"]
)
```

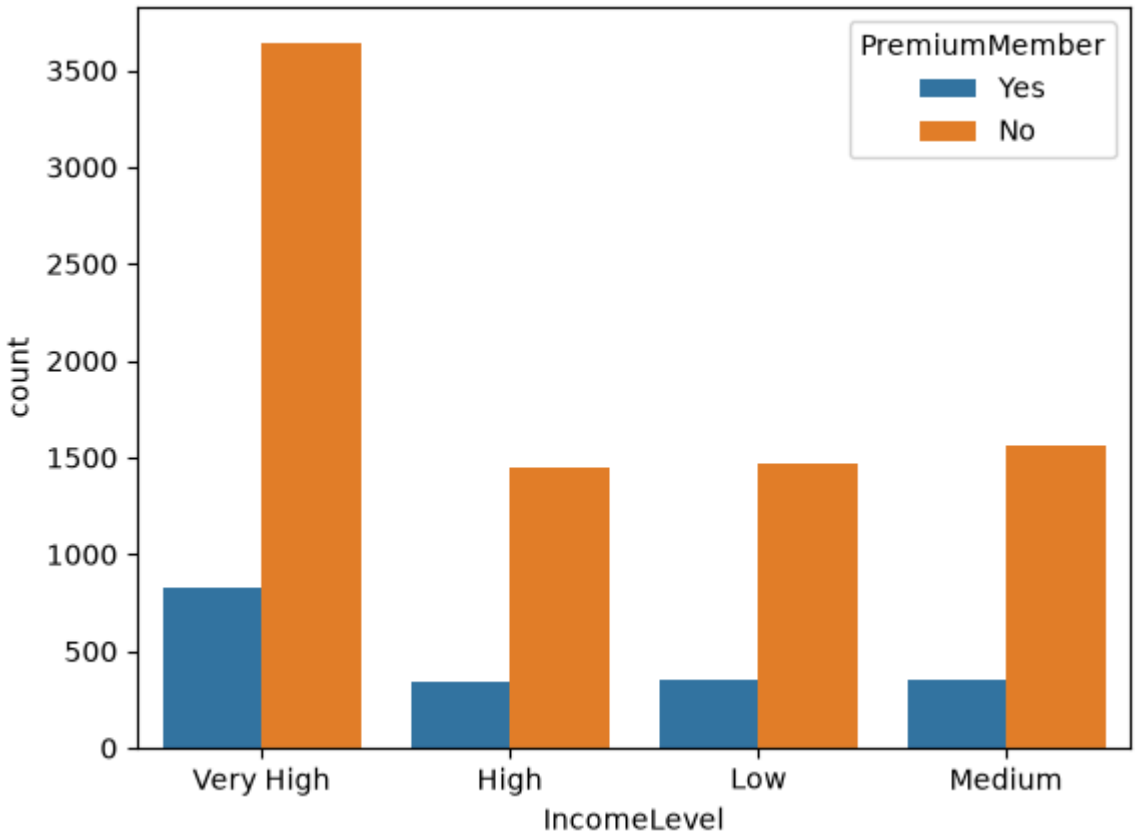
Out[37]:

PremiumMember	No	Yes
Gender		
Female	1507	340
Male	1473	360
Other	3704	809
Prefer not to say	1442	365

10.3.2 Barplot

In [38]:
```
sns.countplot(
    data=df,
    x="IncomeLevel",
    hue="PremiumMember"
```

```
)
plt.show()
```

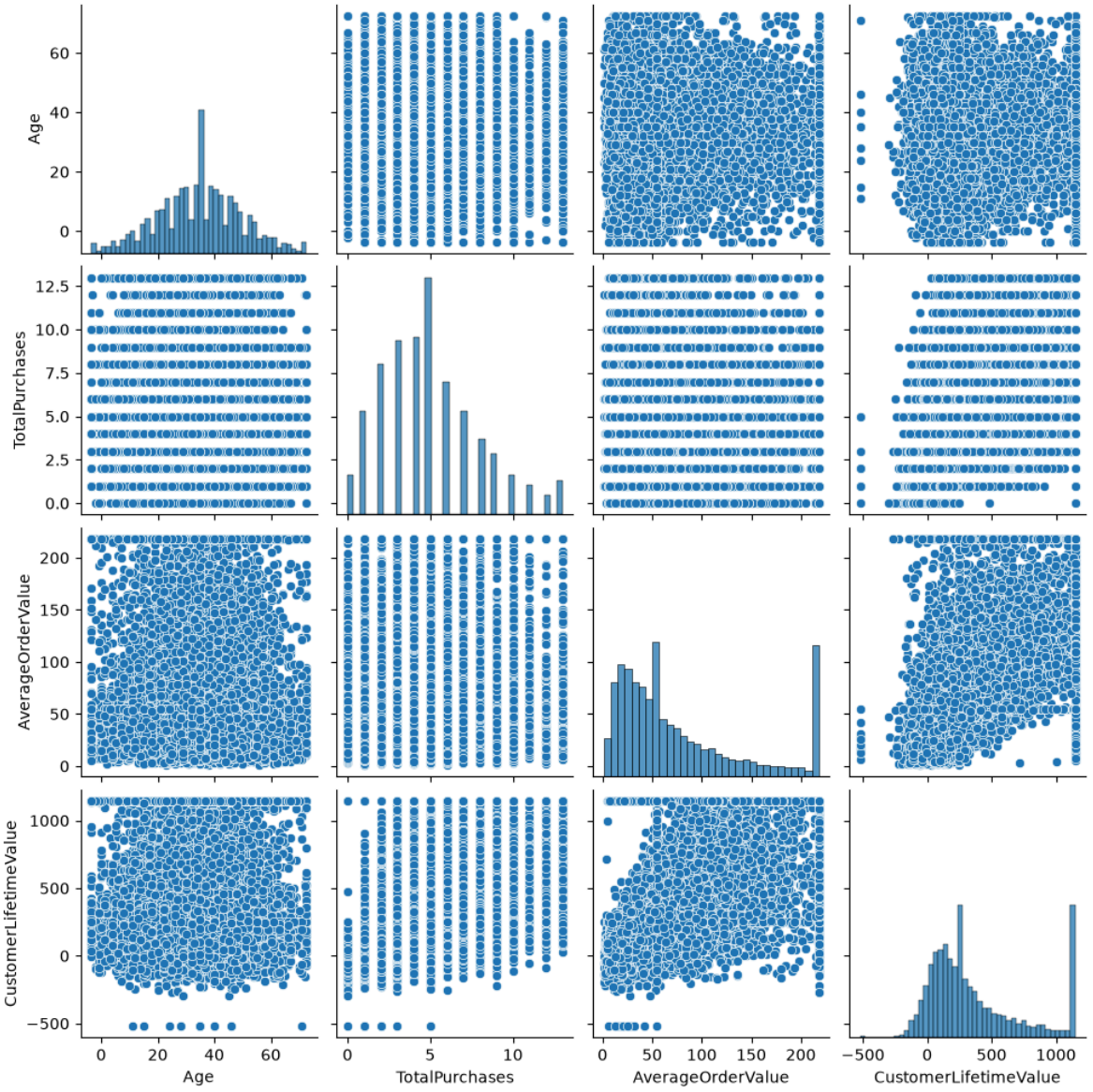


11. Multivariate Analysis

- analysis can be done by more than 2 columns
- Use heat map, pair plot, joint plot

11.1 Pair plot

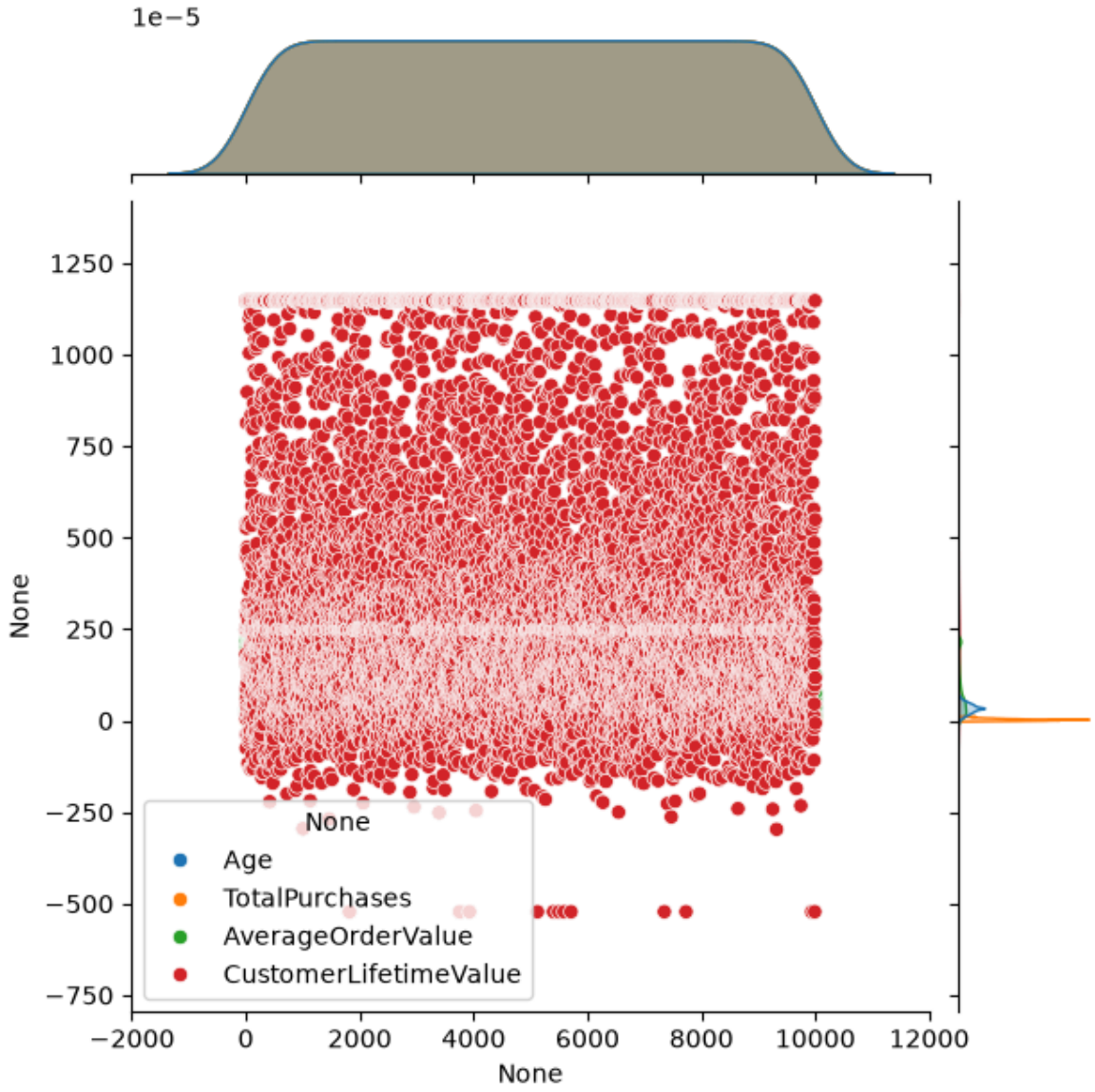
In [40]:
```
sns.pairplot(df[["Age","TotalPurchases","AverageOrderValue","CustomerLife
plt.show()
```



11.2 joint plot

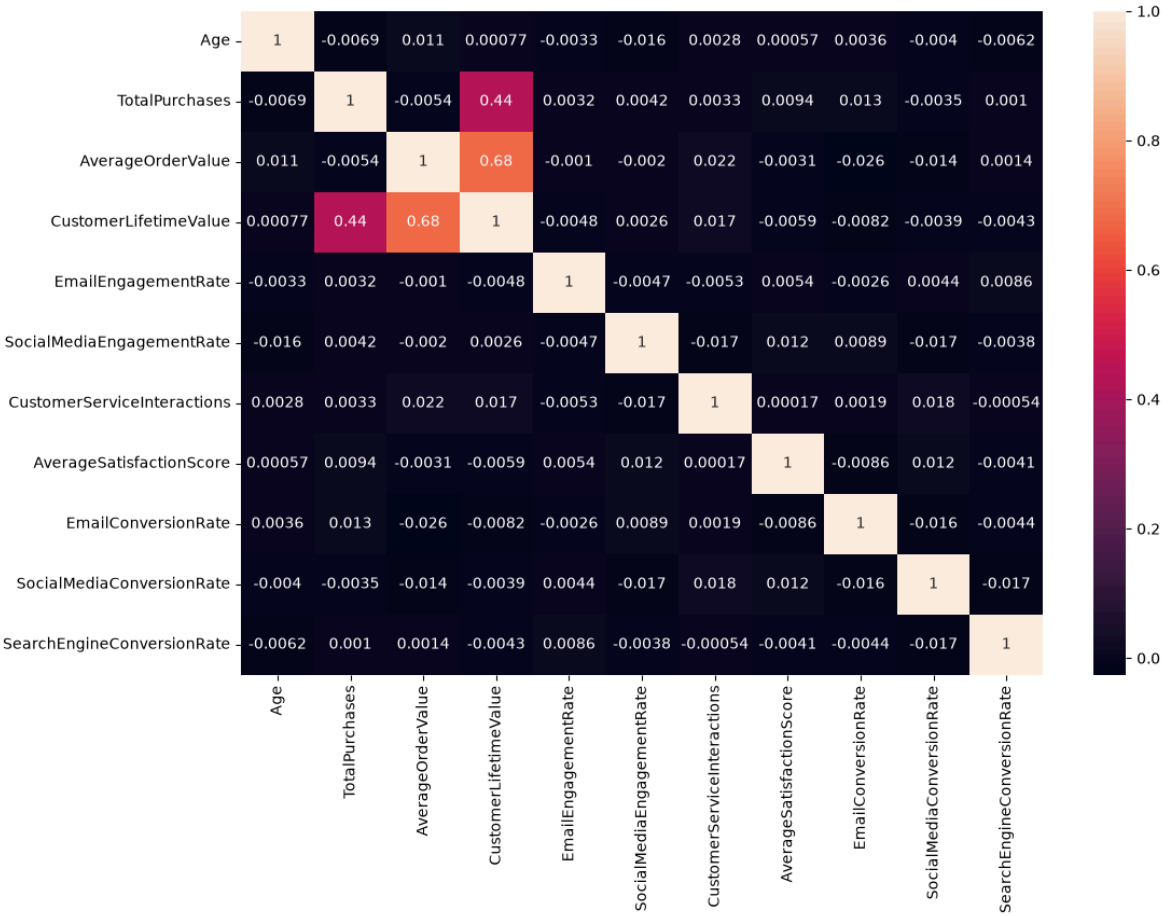
In [46]:
```
plt.figure(figsize = (30,30))
sns.jointplot(df[["Age","TotalPurchases","AverageOrderValue","CustomerLif
plt.show()
```

```
<Figure size 3000x3000 with 0 Axes>
```



11.3 Correlation Heatmap

In [48]:
```
plt.figure(figsize=(12, 8))
sns.heatmap(df[num].corr(),annot=True)
plt.show()
```



12. Business Questions / Business Analysis

1. Which gender has more customers?

In [49]:
```
df["Gender"].value_counts()
```

Out[49]:
```
Gender
Other                4513
Female               1847
Male                 1833
Prefer not to say    1807
Name: count, dtype: int64
```

2.Customer Distribution by Country

In [54]:
```
df["Country"].value_counts()
```

Out[54]:
```
Country
France       1720
Japan        1220
USA          1200
Australia    1194
UK           1177
Other        1171
Canada       1169
Germany      1149
Name: count, dtype: int64
```

3. What is the average Customer Lifetime Value for each gender?

In [50]:
```
df.groupby("Gender")["CustomerLifetimeValue"].mean()
```

Out[50]:
```
Gender
Female              352.664668
Male                362.124239
Other               359.168188
Prefer not to say   366.064888
Name: CustomerLifetimeValue, dtype: float64
```

4.Do Premium Members make more purchases than non-Premium Members?

In [51]:
```
df.groupby("PremiumMember")["TotalPurchases"].mean()
```

Out[51]:
```
PremiumMember
No     5.002584
Yes    4.990395
Name: TotalPurchases, dtype: float64
```

5. Is there a relationship between Total Purchases and Customer Lifetime Value?

In [52]:
```
df["TotalPurchases"].corr(df["CustomerLifetimeValue"])
```

Out[52]:
```
np.float64(0.43666517633284135)
```

6. Which combination of Gender and Premium Membership has the highest average Customer Lifetime Value?

In [53]:
```
df.groupby(["Gender", "PremiumMember"])["CustomerLifetimeValue"].mean().s
```

Out[53]:
```
Gender             PremiumMember
Prefer not to say  No               366.861677
Female             Yes              366.563949
Male               No               365.620879
Prefer not to say  Yes              362.917030
Other              Yes              362.483080
                   No               358.444174
Female             No               349.528798
Male               Yes              347.817157
Name: CustomerLifetimeValue, dtype: float64
```

7. Which FavoriteCategory has the highest average AverageOrderValue?

In [57]:
```
df.groupby("FavoriteCategory")["AverageOrderValue"].mean().sort_values(as
```

Out[57]:
```
FavoriteCategory
Sports         79.778309
Beauty         77.541193
Food           76.579038
Home Goods     76.404372
Books          76.205073
Clothing       75.623029
Electronics    75.208572
Toys           73.867413
Name: AverageOrderValue, dtype: float64
```

8. Do Premium Members have a different average number of TotalPurchases than non-Premium Members?

In [59]:
```
df.groupby("PremiumMember")["TotalPurchases"].mean()
```

Out[59]:
```
PremiumMember
No     5.002584
Yes    4.990395
Name: TotalPurchases, dtype: float64
```

9. Which country has the highest average CustomerLifetimeValue?

In [60]:
```
df.groupby("Country")["CustomerLifetimeValue"].mean().sort_values(ascendi
```

Out[60]:
```
Country
France       373.299934
UK           368.968047
Canada       366.177441
Other        362.762660
Germany      360.608955
Japan        359.946342
USA          349.955175
Australia    330.755814
Name: CustomerLifetimeValue, dtype: float64
```

10. Is there a relationship between TotalPurchases and CustomerLifetimeValue?

In [61]:
```
df["TotalPurchases"].corr(df["CustomerLifetimeValue"])
```

Out[61]:
```
np.float64(0.43666517633284135)
```

11.Is there a relationship between Age and CustomerLifetimeValue?

In [62]:
```
df["Age"].corr(df["CustomerLifetimeValue"])
```

Out[62]:
```
np.float64(0.0007730561527842104)
```

12.How does TotalPurchases vary across IncomeLevel and PremiumMember?

In [63]:
```
df.groupby(["IncomeLevel", "PremiumMember"])["TotalPurchases"].mean()
```

Out[63]:
```
IncomeLevel  PremiumMember
High         No               4.817241
             Yes              4.816327
Low          No               5.041384
             Yes              4.942529
Medium       No               5.088348
             Yes              5.323034
Very High    No               5.023901
             Yes              4.939541
Name: TotalPurchases, dtype: float64
```

13.How does AverageOrderValue vary across FavoriteCategory and Gender?

In [66]:
```
df.groupby(["FavoriteCategory", "Gender"])["AverageOrderValue"].mean()
```

Out[66]:
```
FavoriteCategory  Gender
Beauty            Female               75.640230
                  Male                 80.049170
                  Other                75.512399
                  Prefer not to say    82.212055
Books             Female               70.557874
                  Male                 74.074956
                  Other                75.895952
                  Prefer not to say    84.642278
Clothing          Female               76.825855
                  Male                 90.411364
                  Other                69.772431
                  Prefer not to say    74.774288
Electronics       Female               68.741469
                  Male                 79.799188
                  Other                77.096783
                  Prefer not to say    72.558426
Food              Female               72.938290
                  Male                 80.253409
                  Other                78.684853
                  Prefer not to say    71.253718
Home Goods        Female               75.572685
                  Male                 71.834963
                  Other                79.896480
                  Prefer not to say    72.983853
Sports            Female               76.472594
                  Male                 81.846683
                  Other                80.753224
                  Prefer not to say    78.666437
Toys              Female               67.862189
                  Male                 73.447724
                  Other                73.769782
                  Prefer not to say    80.810279
Name: AverageOrderValue, dtype: float64
```

CONCLUSION

- What the data represents:
- This dataset represents e-commerce customer information.
- It contains customer details, purchase information, customer value, satisfaction, engagement, and preferences.

- The data helps us understand customer behavior and purchasing patterns.

What we have done

- Checked the data types and structure.
- Handled missing values.
- Checked and treated outliers.
- Removed duplicate values.
- Handled inconsistent data.
- Performed data manipulation using Pandas.
- Performed univariate analysis.
- Performed bivariate analysis.
- Performed multivariate analysis.
- Used different charts to understand the data.
- Created business problems and analyzed them.
- Found useful patterns and relationships in the data.

Improvements:

- Improve customer satisfaction.
- Focus on high-value customers.
- Improve customer retention.
- Provide personalized offers to customers.
- Improve marketing strategies.
- Focus on popular product categories.
- Use customer data for better business decisions.

Final Conclusion:

- Overall, the analysis helped us understand customers, their purchasing behavior, preferences, and satisfaction.
- The results can help the business improve customer experience, marketing, sales, and decision-making.

In [ ]: